Cellular automata and their applications

Jan Helm
Technical University Berlin
Email: jan.helm@alumni.tu-berlin.de

Abstract
This paper presents in Part1 the basic theory of Cellular Automata, and its relation to formal languages and finite automata.
Furthermore in Part2 , it presents calculated application examples and evolving self-modifying Cellular Automata.
In Part3, the scope is extended to multiway systems (transition automata with multiple transition rules), and their basic theory is discussed.
In Part3 chapter 4, the genetic evolution of multiway systems is calculated and discussed in the example of the genetic algorithm with goal=entropy applied to a string-replacement three-components multi-rule MCA.

Contents

Part 1 Theory of Cellular Automata

1 Basics

Principle of discrete deterministic Cellular Automata (CA):

▪ *discrete n-dimensional lattice*

CA has a n-dimensional array of cells, usually with finite number in each dimensions, e.g. a 2-dimensional CA has a $n_1 x n_2$ array of cells.

▪ *discrete states*

At each time step $t=1,...N$, each cell is in only one state, $\sigma \in \Sigma$, where Σ is finite .

▪ *local interactions*

Each cell transition depends only on its local neighborhood with range r (usually $r=1$).

▪ *discrete dynamics*

There is a set of rules, determining the transition of a cell $c(t-1) \rightarrow c(t)$

$c(t)= \phi(c(t-1), n_i(t-1))$

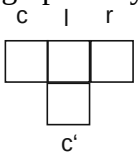
depending on the values of the cell c and the neighbors n_i at time $t-1$.

1.1 The Wolfram Classification Scheme

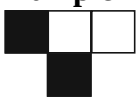
1-dimensional CA

For 1-dim CA: configuration= 3-tuple (c,l, r), new value= c', so rule = ($((c_1,l_1, r_1),c_1')$,,..) , where $(c_1,l_1, r_1),c_1')$ represents the transition $(c_1,l_1, r_1) \rightarrow c_1'$. With 3-tuple configuration there are $2^3=8$ states, so a rule can be represented by a 8-bit number,

graphically



Example

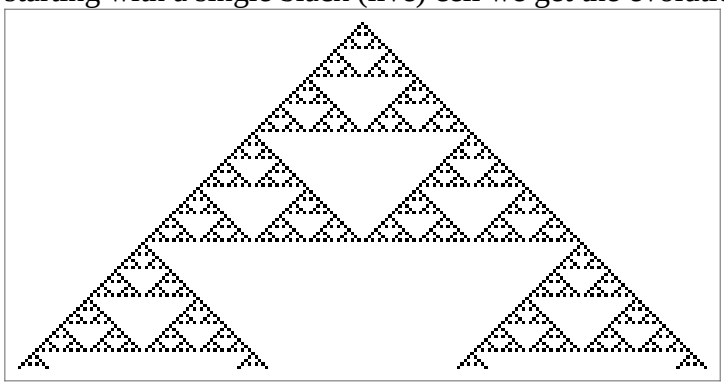


means cell=1, left neighbor l=0, right neighbor r=0 → new cell c'=1

transition (1,0,0) →1

rule 50 "fractional CA" is represented 

starting with a single black (live) cell we get the evolution for 100 steps



2-dimensional CA

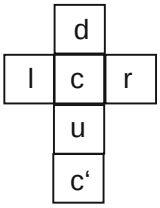
-2-dimensional 5-bit

configuration= 5-tuple (l, d, c, r, u) with neighbor denominations l=left, d=down, c=cell, r=right, u=upper, so rule = ($((l_1, d_1,c_1,r_1, u_1),c_1')$,,..) , where

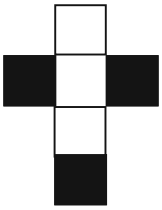
$((l_1, d_1,c_1,r_1, u_1),c_1')$ represents the transition $(l_1, d_1,c_1,r_1, u_1) \rightarrow c_1'$.

With 5-tuple configuration there are $2^5=32$ states, so a rule is represented by a 32-bit number,

graphically

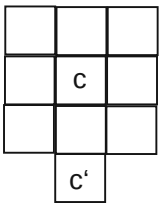


Example



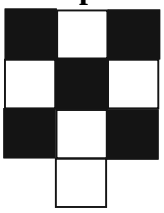
means horizontal neighbors=1, cell=0 → new cell c‘=1

-2-dimensional 9-bit outer totalistic (depends only on the number of live neighbors)
configuration= 2-tuples (n, c) where n=number of live (=1) neighbors, so rule = (((n_1,c_1),c_1’),,..) , , where ((n_1,c_1),c_1’) represents the transition (n_1,c_1)→ c_1’ .
With this configuration n is the range (1, 8), so there are 2*8=16 states, so a rule is represented by a 16-bit number.
graphically



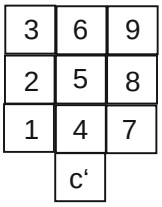
is a 9-bit configuration with 8 neighbors and old cell c, new cell c’

Example



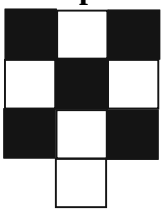
represents the rule with n=4, c=1, c‘=0, and the transition (4,1)→ 0 .

-2-dimensional 9-bit (8 neighbors+cell)



is a 9-bit configuration with 8 neighbors and old cell c, ordered as shown on the left, new cell c’

Example



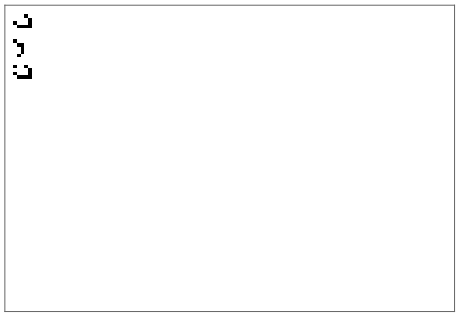
represents the rule with alive neighbors (n_1,n_3,n_4,n_7,n_9), c=1, c‘=0,
and the transition (101010101)→ 0

-boundary condition
CA has a finite defined region *D*, for a 1-dim. CA $D=[-n,n]$ is a symmetric interval around the origin, for 2-dim. CA $D=(x\in[-n,n],\ y\in[-n,n])$ is a square with the origin in the center.

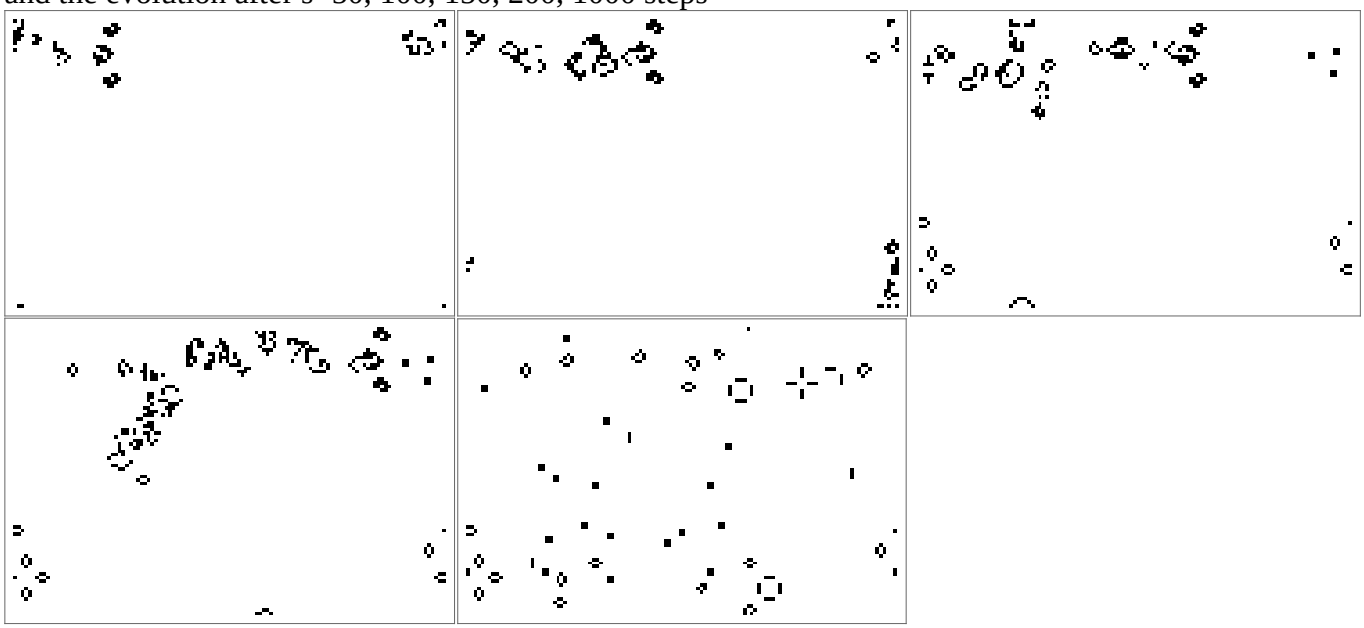
For cells at the boundary, e.g. at the left edge in the example above with cell position $(x(c) = -n,\ y(c) = 0)$, the values of the neighbors outside *D* have to be set. There is the *fixed-value* convention (outside neighbors=0), and the *cyclic* convention (outside neighbors take values from the opposite boundary in *D*).
In Wolfram's implementation, which is adopted here, the valid boundary values are those of the cyclic convention.

1.2 Game of Life
The most famous 2-dimensional CA is Conway's Game of Life (2-dimensional 9-bit outer totalistic CA), with the rules
c=1, n=(2,3) → c'=0
a live cell with more than 2 neighbbors dies (overpopulation)
c=0, n=3 → c'=1
a dead cell with 3 neighbors becomes alive (regeneration)
else c'=0

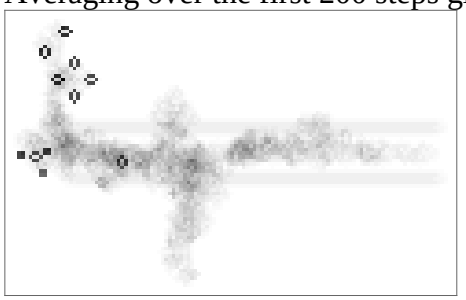
The Wolfram rule number of Game of Life is 224.
"train" is a famous scenario here with a field of 80x120 and the initial configuration



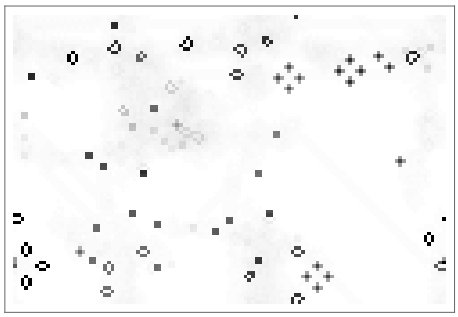
and the evolution after s=50, 100, 150, 200, 1000 steps



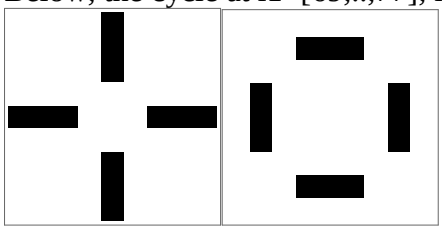
Averaging over the first 200 steps gives the image,



averaging over 1000 steps gives the image,

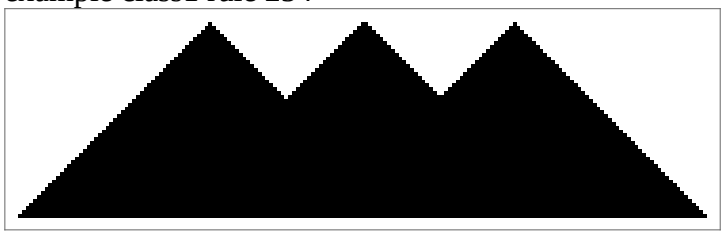


here the black fields in the left lower and right lower corner are constants of the CA scenario, the dark-grey cross-structures on the upper right and the lower right are cycles.
Below, the cycle at i1=[69,..,77], i2=[81,..,89] with the period 2 is shown.

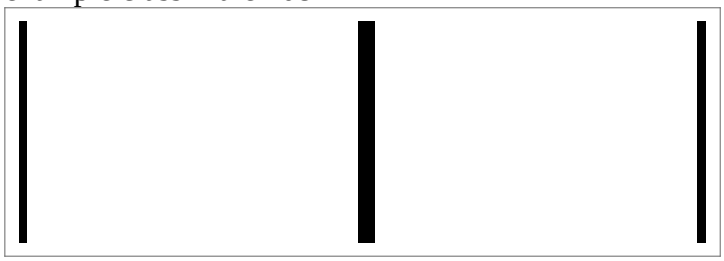


1.3 Classification of 1-dimensional CA

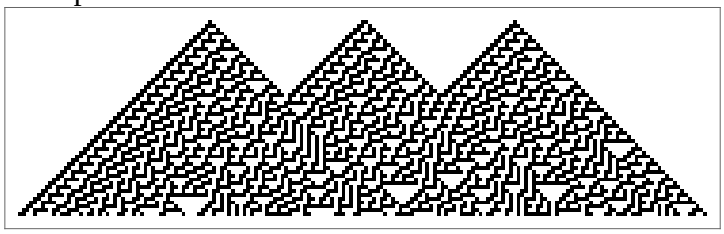
1-dimensional CA can be characterized by four classes, where the following class encompasses the preceding and is significantly more complex. The same scheme applies to 2-dimensional CA, but here higher classes have a higher percentage than with 1-dimensional CA.
Class1 rules produce uniform configurations.
example class1 rule 254



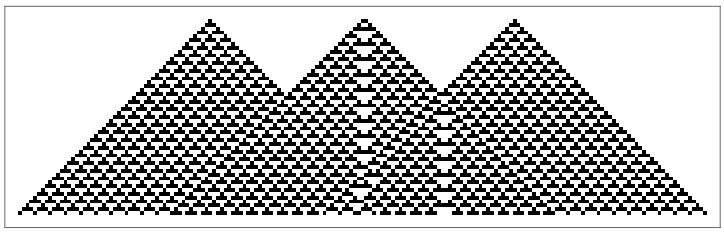
Rules in Class2 produce a uniform final pattern or cycles.
example class2 rule 108



The configurations produced by Class3 rules are mostly random, with some regular fractal patterns.
example class3 rule 30



Class4 rules produce localized, stable, but non-periodic patterns, some propagating.
example class4 rule 54



Conjecture by Wolfram: class4 CA have the computability power of Turing machines

The effect of small changes in initial configurations are
class1. no change in final state
class2. changes only in a finite region
class3. changes in a region of increasing size
class4. changes irregularly

2 CA , languages, finite automata

A CA generates sets of configurations with time. Such a set of strings is described as formal language. The language is formed from a sequence of symbols according to definite grammatical rules (grammar).
The language is recognized (parsed) by a finite automaton, in the same sense as a parser for the computer language C recognizes correct C commands, and generates a syntax tree.
These finite automata are specified by finite state transition graphs; words in a language correspond to possible paths through the graph.
It is always possible to **find a simplest finite automaton**, or grammar, which correspond to a particular regular language. This simplest finite automaton provides a minimal representation for sets generated by a CA, and its size (=number of states) is a measure of their complexity.
Result suggest the general result that the **language entropy is non-decreasing** with time for all CA. It gives a first indication of a generalization of the second law of thermodynamics to irreversible systems.

2.1 CA generated languages

Regular language and class1,2 CA

The sets of strings $\Omega^{(t)}$ generated by (t= number of steps) in the evolution of a class1 or class2 CA form a regular language (definition see below).

Regular language

A regular language over a given alphabet, Σ is defined recursively as follows:
- The empty string belongs to the language.
- For every $a \in \Sigma$ the string a belongs to the language.
- Closure under union, intersection, concatenation, repetition, complement belongs to the language

Example: strings (01)*11(10)*

Context-free languages and class3 CA

Context-free languages are generalizations of regular languages (definition see below).
Words in context-free languages may be viewed as sequences of terminal nodes (leaves) in trees constructed according to context-free grammatical rules. These rules are production (replacement) rules in the form string1 → string2 , where strings are lists of (non-terminal) variables (e.g. <expression>) and (terminal) atoms (e.g. + , 10).
Class3 CA generate sets of strings, which (mostly) form a context free language.

Context-free language

A context-free grammar can be described by a four-element tuple (V,Σ,R,S), where
- V is a finite set of variables (which are non-terminal);
- Σ is a finite set (disjoint from V) of terminal symbols=atoms ;
- R is a set of production rules where each production rule maps a variable to a string $s \in (V \cup \Sigma)^*$
- S (which is in V) which is a start symbol.

Example: arithmetic expressions
- Start symbol = <expression>
- Terminal symbols = {+,−,∗,/,(,),number}, where “number” is any number
- Production rules:

<expression> → number
<expression> → (<expression>)
<expression> → <expression> + <expression>
<expression> → <expression> - <expression>
<expression> → <expression> * <expression>
<expression> → <expression> / <expression>

CA and unrestricted languages and UTM, class4 CA

CA in class4 generate sets of strings, which in general form a unrestricted language.

Unrestricted languages (i.e. sets of algorithmically computable strings from a certain alphabet) are associated with universal computers. According to the Church-Turing-thesis, such a set is identical with the set of strings produced by a universal Turing machine (UTM).
UTM is capable of "universal computation" , i.e. fed with an appropriate program it can simulate the behavior of any other computational system.
There are several quite different systems, which are computationally universal.
Among these are:
string manipulation systems which directly apply the production rules of type 0 languages (recursively enumerable)
von-Neumann computers with infinite precision
mathematical systems such as Kleene's λ-calculus (=general recursive functions).

Some CA have also been proved computationally universal.
For example, a one-dimensional cellular automaton with k = 18 colors and range r = 1 is equivalent to the simplest known universal Turing machine [2] [6].
The two-dimensional CA Game of Life, with k = 2, r=1, number of neighbors N_n=9 has also been proved computationally universal [2]. It is conjectured that all class4 CA are computationally universal [2].

2.2 Chomsky hierarchy of languages
The following table Chomsky's five types of grammars, the class of generated language, the type of analyzing automaton, and the form of its production rules.

Grammar	Language type	Automaton	Production rule	Example
type0	recursively enumerable	Turing machine TM	$\gamma \rightarrow \alpha$	output(TM)
type01	recursive	halting TM (hTM)	$\gamma \rightarrow \alpha$ recursive	logic expressions(Presburger arithmetic)
type1	context-sensitive	lin.-bounded, non-det. TM	$\alpha A \beta \rightarrow \alpha \gamma \beta$	$L=\{a^n b^n c^n \mid n>0\}$ natural lang.
type2	context-free	non-det. stack automaton	$A \rightarrow \alpha$	$L=\{a^n b^n \mid n>0\}$ computer lang.
type3	regular	finite state automaton	$A \rightarrow a$ $A \rightarrow aB$	$L=\{a^n \mid n>0\}$ arithmetic expressions

Meaning of symbols:
a = terminal
A = non-terminal
α, β, γ = string of terminals and/or non-terminals
α, β =maybe empty
γ = never empty
Presburger arithmetic is the first-order theory of the natural numbers with addition (but without multiplication).

2.3 Complexity classes
Complexity class *P* (polynomial) contains all decision (yes/no) problems that can be solved by a deterministic Turing machine during a polynomial computation time $t_s \sim t^n$.
Complexity class *NP* (non-deterministic polynomial) is determined by a non-deterministic Turing machine (several next states in one operation, performed in parallel) (NTM) that runs in polynomial time $t_s \sim t^n$.
Clearly P $\subseteq$ NP, and there is much evidence that P $\neq$ NP.
The complexity class NP is related to the complexity class *co-NP* for which the answer "no" reached in polynomial time. Whether or not NP = co-NP is an open question .
The complexity class NL (**N**ondeterministic **L**ogarithmic-space) is the complexity class of decision problems solvable by a nondeterministic Turing machine with a logarithmic memory space $M_s \sim Log\, t$.

The complexity class PSPACE is the set of all decision problems that can be solved by a TM using a polynomial memory space $M_s \sim t^n$.

The complexity class EXPTIME is the set of all decision problems that can be solved by a TM in exponential computation time $t_s \sim 2^{t^n}$.

We have the containment relations

$NL \subseteq P \subseteq NP \subseteq PSPACE$

$NL \subseteq PSPACE \subseteq EXPTIME$

$P \subseteq EXPTIME$

3. CA configuration sets

3.1 Finite time sets of CA and corresponding languages

$\Omega^{(t)}$ is the set of configurations generated after t time steps of CA evolution, starting from the set $\Omega^{(0)} = \Sigma$.

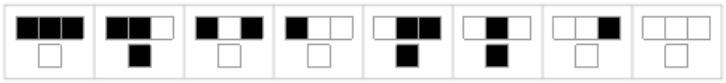
$\Omega^{(t)}$ is a regular language for class1 and class2 CA, generated by a corresponding set of production rules, and a corresponding finite state automaton (FNA).

Examples

-CA76 with totalitaristic rule 76, k= 2 colors, range r=1 [3]

Rule 76 , 2^3=8bit: k=2, r=1, transition (L,c,R)→c', 4 states s=(u0=00,u1=01,u2=10,u3=11) =neighbors(L,c), next state bei (L,c,R)→c' s'=(c,R)

rule 76: totalitaristic: Sn=2→c'=0 , Sn=1→c'=c , Sn=0→c'=c, where Sn=number live neighbors

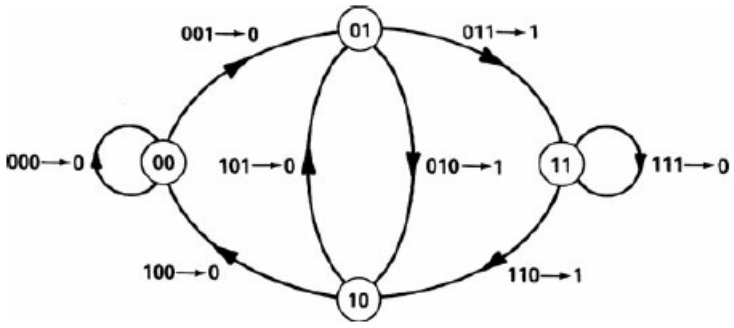


$$111 \to 0,\ \ 110 \to 1,\ \ 101 \to 0,\ \ 100 \to 0,\ \ 011 \to 1,$$
$$010 \to 1,\ \ 001 \to 0,\ \ 000 \to 0.$$

production set

$$u_0 \to 0u_0,\ \ u_0 \to 0u_1,\ \ u_1 \to 1u_2,\ \ u_1 \to 1u_3,\ \ u_2 \to 0u_0,$$
$$u_2 \to 0u_1,\ \ u_3 \to 0u_3,\ \ u_3 \to 1u_2.$$

The configuration set $\Omega^{(1)}$(CA76) after t=1 steps of the 1-dimensional CA rule 76 with k=2 colors (0,1) and range r=1 (2 neighbors) is described by the non-deterministic **finite automaton (NDFA) NDFA76** depicted below.



Generated strings are represented by possible paths through the graph.

The nodes (states) in the graph are labelled by pairs s=(L,c) in the transition (L,c,R)→c' , the next state is s'=(c,R) .

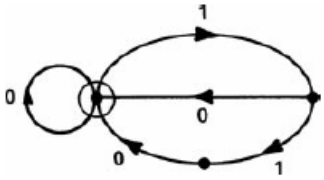
There are 2 paths u0→u0 without repetition

p1=(u0, u1, u2, u0) producing string s1=0^+10

p2=(u0, u1, u3, u2, u0) producing string s2=0^+110

so the **produced one-step-stri ng set** is $\Omega^{(1)} = \left(0^*\right)1\left(0 \vee 10\right)$.

The set $\Omega^{(1)}$ can be produced by the **minimal deterministic automaton (DFA) mDFA76** with 3 states



Let us calculate the set $P(u_0)$ of **all produced strings from mDFA76**. From the production rules we get the set equations for the sets produced from a state $P(u_0)$, $P(u_1)$, $P(u_2)$, $P(u_3)$:

$P(u_0)\ P(u_1) = (1P(u_2)) \cup (1P(u_3))$

$P(u_2) = (0P(u_0)) \cup (0P(u_1))$

$P(u_3) = (1P(u_2)) \cup (0P(u_3))$

From the first equation follows that $P(u_0)$ must start with 0 : $P(u_0) = 0^+ Q(u_0)$ and furtheron

$P(u_3) = (10^{2+} Q(u_0)) \cup (0^+)$

$P(u_1) = (10^{2+} Q(u_0)) \cup (10P(u_1)) \cup (110^{2+} Q(u_0)) \cup \{10^+\}$

$P(u_1) = (10^{2+} Q(u_0)) \cup (10P(u_1)) \cup (110^{2+} Q(u_0)) \cup (1010^{2+} Q(u_0)) \cup (10110^{2+} Q(u_0)) \cup \{1010^+\} \cup \{10^+\}$

$Q(u_0) = P(u_1) \cup Q(u_0)$, so $P(u_1) \subseteq Q(u_0)$ and

$P(u_0) = 0^+ Q(u_0) = 0^+ C(10^+, 1010^+, 10^{2+}, 110^{2+}, 1010^{2+}, 10110^{2+})$, where C means: concatenation with repetitions. The minimal length term in $P(u_0)$ is 010 , which corresponds to the path p1 in $\Omega^{(1)}$.

The state transition graph for mDFA76 that generates the regular language $\Omega^{(1)}$ has the transition matrix

$$M = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

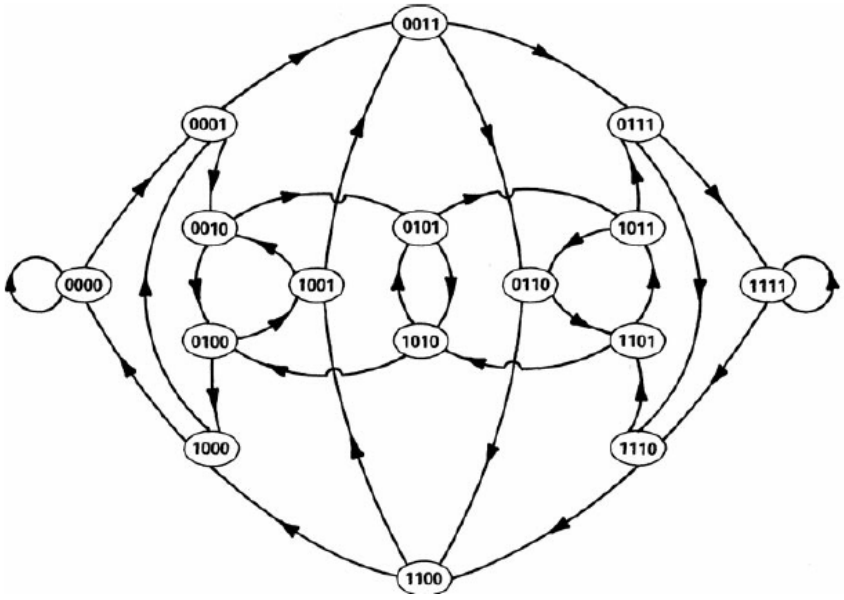
its characteristic polynomial $\chi(\lambda) = 1 + \lambda + \lambda^2 - \lambda^3$

The elements of M^X give the numbers $N(X)$ of possible length X paths in M^X. For lengths from 1 to 10 these numbers for mDFA76 are: 2, 4, 7, 13, 24, 44, 81, 149, 274, and 504. In general, at least for large X,

$$N(X) \simeq \mathrm{Tr}[M^X] = \sum \lambda_i^X \sim \lambda_{\max}^X,$$

where the λ_i are the eigenvalues of M, and λ_{max} is the largest of them [2], for mDFA76 $\lambda_{max} = 1.38929$.

-CA2r76 same rule with k= 2 range r=2 (4 neighbors) [3]
Rule 2r76: totalitaristic: Sn=4→c'=0 , else →c'=c, where Sn=number live neighbors
Rule 2r76= 1894838512 [3] ,
with 2^5=32bit k=2, r=2, transition (L1,L2,c,R1,R2)→c' , 2^4=16 states s=(u0=0000,u1=0001,...,u5=1111) =neighbors(L1,L2,c,R1), next state s'= neighbors(L2,c,R1,R2)



NDFA2r76 for rule 2r76 case $k = 2$, $r = 2$

3.2 Legal finite time sets

We consider as a sample set the 32 legal cellular automaton rules with $k = 2$ and $r = 1$.
A rule φ is considered legal if it is symmetric, and maps the null configuration (with all site values 0) to itself. Each of the 256 possible $k = 2$, $r = 1$ cellular automaton rules is conveniently labelled by a "rule number," defined as the decimal equivalent of the sequence of binary digits φ[1, 1, 1], φ[1, 1, 0],⋯, φ[0, 0, 0] .
The total number of states $\Xi^{(t)}$ in the minimal DFA that generates a set $\Omega(t)$ provides a measure of the complexity of the set $\Omega^{(t)}$ considered as a regular language. $\Xi^{(t)}$ gives the size of the shortest specification of the set $\Omega^{(t)}$ in terms of regular languages: this shortest specification becomes longer as the complexity of $\Omega^{(t)}$ increases.

Characteristic polynomials $\chi^{(1)}(\lambda)$ for the adjacency matrices of state transition graphs for minimal DFA representing regular languages generated after one time step in the evolution of legal $k = 2$, $r = 1$ cellular automata. The nonzero roots of these polynomials determine the **number of distinct symbol sequences** that can appear in configurations generated by the cellular automaton evolution. The maximal root λ_{max} determines the **limiting entropy** of the sequences.

Rule	$\chi^{(1)}(\lambda)$	λ_{max}
0	$1-\lambda$	1.000
4	$-1-\lambda+\lambda_2$	1.618
18	$(1-\lambda-\lambda^2)(-1+\lambda-2\lambda^2+\lambda^3)$	1.755
22	$\lambda(1-\lambda)(2-2\lambda^2+6\lambda^3-3\lambda^4-5\lambda^5+10\lambda^6-5\lambda^7-3\lambda^8+6\lambda^9-2\lambda^{10}+2\lambda^{11}-3\lambda^{12}+\lambda^{13})$	1.917
32		1.618

In all the cases given, $\Xi(t)$ is seen to be non-decreasing with time. Cellular automata with only class 1 or 2 appear to give $\Xi(t)$ which tend to constants after one or two time steps, or increase linearly or quadratically with time. Class 3 and 4 cellular automata usually give $\Xi(t)$ which increase rapidly with time. In general, $1 \leq \Xi^{(t)} \leq 2^{k^{2rt}} - 1$.
The characteristic polynomials $\chi(\lambda)$ such as those in Table 3.2 obtained from regular languages transition matrices are always monic (the term with the highest power of λ that appears in them always has unit coefficient). The largest roots λ_{max} of the $\chi(\lambda)$ for regular languages are thus always algebraic integers, so that the entropies for regular languages are always the logarithms of algebraic integers. The minimal polynomial with λ_{max} as a root has a degree not greater than the size $\Xi(t)$ of the minimal DFA for a regular language $\Omega(t)$ This bound is usually not reached, since the characteristic polynomial $\chi(\lambda)$ is usually reducible. Notice that in many cases, $\chi(\lambda)$ has several factors with equal degrees. (The factorizations of the $\chi(\lambda)$ are related to the colouring properties of the corresponding graphs. Note that graphs

corresponding to minimal DFA always have trivial automorphism groups.) Factors (other than λ_n) with smaller degrees appear to be associated with transient subgraphs in the minimal DFA
graph.

3.3 Entropy and fractal dimension of CA

-Measure entropy of CA
Information entropy in Shannon's concept is given by $H = -\sum_i p_i \log_2 p_i$, where p_i is the actual occurrence probability of sequence.

Following Wolfram, we define the measure entropy of CA $S(n) = -\frac{1}{n}\sum_n p_n \log_2 p_n$

for the number $N(n)$ of sequences of length *n*, we have $p_n = \frac{N(n)}{2^n}$ (for binary CA with *k=2* colors), and the measure entropy becomes $S(n) = -\frac{1}{n}\sum_n \frac{N(n)}{2^n}\left(\log_2 N(n) - n\right)$, in the limit we get the characteristic measure *entropy of CA*

$S = \lim\left(-\frac{1}{n}\sum_n p_n \log_2 p_n \, , n \to \infty\right)$.

The Shannon entropy for $N(n)$ of sequences of length *n*, is $H(n) = -\sum_n p_n \log_2 p_n = -\sum_n \frac{N(n)}{2^n}\log_2\left(\frac{N(n)}{2^n}\right)$

-Topological entropy of the configuration set of CA
The topological entropy or fractal dimension of a CA is

$s = \lim\left(-\frac{1}{n}\sum_n N(n) \, , n \to \infty\right)$

The set of infinite configurations $\Omega(t)$ generated by cellular automaton evolution may be considered to form a n-dimensional set. The Hausdorff (fractal) dimension of this set is given by *s* .
For any regular language CA (class1 , class2), this topological entropy *s* is given by λ_{max} (largest eigenvalue of the transition matrix *M*)

$s = \log \lambda_{max}$

E.g. for mDFA76, the topological entropy (=fractal dimension) is thus

$s \simeq \log_2 1.83929 \simeq 0.87915$.

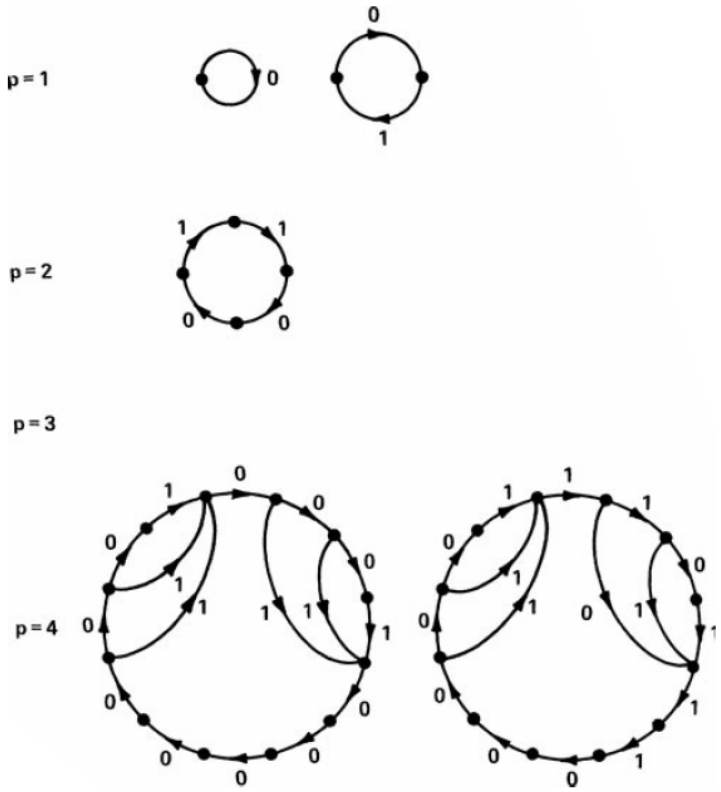
3.4 Periodic sets

A simple class of invariant sets consists of configurations stable in time .
Such sets form finite-complement languages (finite set of fixed blocks of symbols is excluded).
Consider the set of configurations that are periodic) under CA with *k* = 2 and *r* = 1. Such configurations are containing only neighborhoods {*ai*−1, *ai*, *ai*+1} where

$\phi[a_{i-1}, a_i, a_{i+1}] = a_i$.

The complete set forms a finite-complement language, with a maximum distinct excluded block of length 3.

Examples



Minimal deterministic finite automata corresponding to sets of configurations with (temporal) periods exactly p under cellular automaton rules **a** 90, **b** 18 and **c** 22.

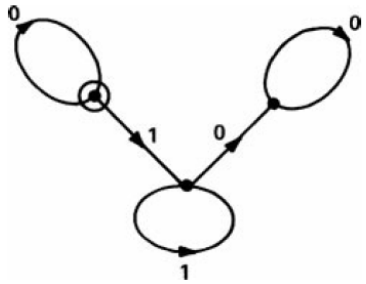
3.5 Limit sets

Class3 CA generate in the time limit (step number t→∞) languages, which are non-regular, but belong to the context-free languages.

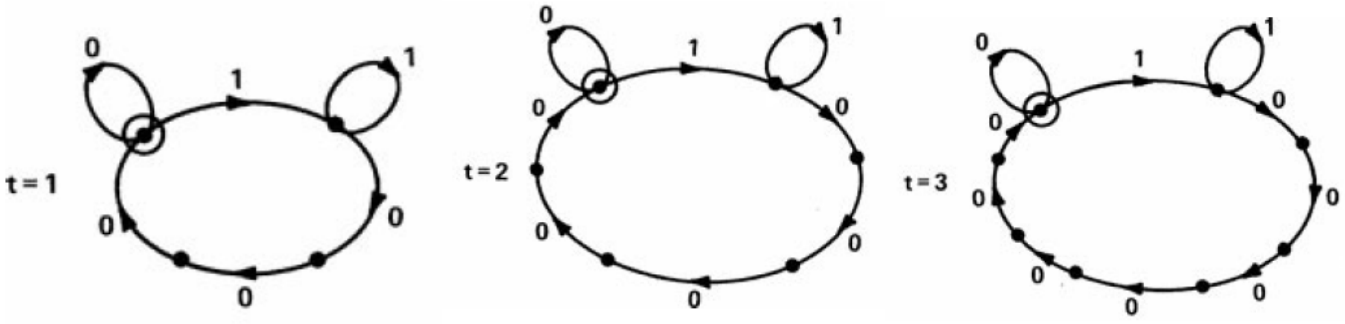
For example, there are CA with k=3 colors (symbols), which after t time steps generate configurations consisting roughly of sequences

$(10^j 20^j 1) \quad j \leq t.$ which belong to a context-free (non-regular) language.

Examples



DFA representing the limit set for CA rule 128. The generic expression for the limit set is $0^*11^*00^*$ which belongs to a context-sensitive language (see Chomsky-hierarchy).



Minimal deterministic finite automata (mDFA) corresponding to the regular languages $\Omega^{(t)}$ generated by CA rule 128. The mDFA have the same structure, increasing size. They correspond to finite complement languages (closed under complement operation), with patterns $10^j\,1\,10^j1$ excluded for $1 \le j < 2t$.

4 Computability

The limiting properties (i.e. in time limit step number t→∞) of a CA are determined by an infinite computational process.
For sufficiently simple CA, of class1 and class2, it is possible to give a finite specification of their limiting properties (regular language).
For most class3 and class4 cellular automata, there is no such finite specification.
CA of class3 show basically chaotic or fractal behavior in the limit, they generate in general context-free or context-sensitive languages
Undecidability and non-computability (e.g. Turing halting problem of T(n)) are features of infinite processes, and they apply to CA of class4.

4.1 Chaitin omega number

The best-known example of an uncomputable number is Chaitin's omega number Ω_U .

Chaitin omega number Ω_U , is the halting probability of a prefix-free (self-delimiting) universal Turing machine (UTM). Every Chaitin constant is simultaneously computably enumerable (= increasing, converging sequence of rationals), and algorithmically random (= binary expansion is an algorithmic random sequence), therefore uncomputable (Chaitin 1975).
A Chaitin's constant Ω_U can be defined as

$$\Omega_U = \sum_{P(U),U\ halts} 2^{-|P(U)|}$$

which gives the probability that for any program *P(U)*, for a particular prefix-free (self-delimiting) UTM *U* will halt, where $|P(U)|$ =length(*P(U)*).

A set of strings is prefix-free, if there is no non-empty initial string in common for all strings in the set.

Theorem: the **bits of Ω are logically irreducible**, they cannot be obtained from axioms simpler than they are or in other words **Ω is algorithmically incompressible**.
[G. Chaitin, Proving Darwin, Pantheon Books, New York 2012]

Calude *et al.* (2002) computed the first 64 bits of Chaitin's constant Ω_U for a certain universal Turing machine .
[Cristian S. Calude. *Information and Randomness: An Algorithmic Perspective*, Springer 2002]

$\Omega_{U2}^{32} = 0.00010000000100001010011101000100$ U2 in

4.2 Computability of CA and physical systems

The behavior of a pattern in a CA with a number of sites N with initial local (bounded) seed can always be determined in at most $L=k^N$ steps, in a system with complexity *S* (Kolmogorov complexity in CA or equivalently Boltzmann-Shannon entropy in a physical system) ,
the **computation time** (number of steps) becomes $\boldsymbol{L=k^S}$, for standard binary CA: $\boldsymbol{L=2^S}$.
For example, the computation time for **ideal gas with *N* particles** with Boltzmann entropy (Sackur-Tetrode equation)

$$S = N\,k_B\,Log\left(\left(\frac{T}{T_0}\right)^{c_V}\left(\frac{V}{V_0}\right)\right),$$

where c_V =dimensionless specific heat capacity, *T*=temperature, *V*=volume ,

becomes $L = 2^{N\,Log\left(\left(\frac{T}{T_0}\right)^{cV}\left(\frac{V}{V_0}\right)\right)}$ or $\boldsymbol{T\sim 2^N}$.

The behavior of a pattern in a CA from initial arbitrary large seeds is more complex.
The value $a^{(t)}$ of a site after *t* steps depends in general on $n(t) = 2\lambda\, t \le r\, t$ initial site values, where λ is the Lyapunov exponent and *r* = range (effective neighbor distance) .

The **Lyapunov exponent** of a dynamical system is a quantity that characterizes the rate of separation of infinitesimally close trajectories.
Quantitatively, two trajectories in phase space with initial separation vector δZ_0 at the rate

$$|\delta \mathbf{Z}(t)| \approx e^{\lambda t} |\delta \mathbf{Z}_0|$$

where λ is the Lyapunov exponent. Here the distance in phase space is measured as the number of elementary cells in phase space, in CA it is accordingly the number of separating CA cells .
In class1 and class2 CA the computation time is $L \sim Log\, t$, for class3 and class4 CA it becomes $L \sim t$.
The interesting question is the behavior of a pattern string of length *n* after time *t* , and for class4 CA we have a NP-complex problem, and we have to test all $k^{n+2\lambda t}$ candidate initial strings, so the computation time becomes $L \sim k^{n+2\lambda t}$,
or for standard binary **class4 CA** we get $\boldsymbol{L \sim 2^{n+2\lambda t}}$.

4.3 Algorithmic randomness
There are many simple algorithms, which produce almost random (correlation values below certain limit) binary strings.
Examples are
▪ Irrational numbers
The irrational number $\sqrt{2}$ has a binary digit sequence, which has no recognizable regularity with current algorithms within a reasonable time, so it is a very good random approximation, although it is generated by the root-algorithm, so it is certainly not purely random.
▪ Integer random generators
These are common (pseudo-) random generators of the form *x′ = ax + b mod n* , used in computer programs. They are also very effective, although certainly not pure random, for the same reason as for irrational numbers: they are algorithmic.

There are several reasons for stochastic behavior in CA
▪ Random initial state (homoplectic behavior)
Here the initial configuration (initial value string) is random (Martin-Löf negative) or almost random, this can produce temporary (time *t*<*n*) or permanent (in time limit) almost random value strings.
▪ Random rules in CA (stochastic)
Here one or several rules have multiple transitions , which are taken with probabilities $p_{i,1},...,p_{i,n}$

$$R_i = \left((N_i, c_i) \to c'_{i,1}(p_{i,1}), (N_i, c_i) \to c'_{i,2}(p_{i,2}), ..., (N_i, c_i) \to c'_{i,n}(p_{i,n})\right) ,$$

the CA is a stochastic CA.
▪ Deterministic chaotic CA (autoplectic behavior)
CA of class3 are capable of generating (almost) random strings , with non-random simple initial configuration. This is called autoplectic behavior.

Among the autoplectic 1-dimensional CA the simplest random generators are those nonlinear mod(2), for example

$$a_i' = \left(a_{i-1} + a_i + a_i a_{i+1}\right) \bmod 2 \quad \text{rule 30}$$

$$a_i' = \left(1 + a_{i-1} + a_{i+1} + a_i a_{i+1}\right) \bmod 2 \quad \text{rule 45}$$

The entropy of a string is basically the number of possible sequences that occur.
The entropy of a binary string *s* is
Kolmogorov entropy $K(n) = N(n)$, where *N(n)* is the number of distinct length *n* blocks in the string
Shannon entropy $S(n) = -\sum_{i=1}^{N(n)} p_i \log_2 p_i$, where p_i is the probability of the *i-th* block. According to the Shannon-Hartley theorem, the minimum length (=*compressibility*) of a block with occurrence probability p is $n_{min} = -\log_2 p$.
The two entropies are equal in the limit of large *n* .
For random strings, we get with $p_i = 2^{-n}$ both entropies are maximal and equal

$K(n) = n \quad S(n) = -\sum_{i=1}^{2\wedge n} p_i \log_2 p_i = n$

For CA the relative entropies are used

$K_r = \lim_{n\to\infty} \frac{N(n)}{n}$

$S_r = \lim_{n\to\infty} -\frac{1}{n} \sum_{i=1}^{N(n)} p_i \log_2 p_i$

For CA, we introduce the relative Shannon entropy on space-time patches $X \times T$

$h_r = \lim_{X\to\infty} \lim_{T\to\infty} -\frac{1}{T} \sum_{i=1}^{2\wedge XT} p_i \log_2 p_i$

E,g. for the rule 30 random generator we get after some calculation $h_r \le 1.2$

The rate of change transmission to the left and to the right of CA is determined by the Lyapunov exponent λ_L and λ_R .

E,g. for the rule 30 random generator we get $\lambda_L = 0.242$ and $\lambda_R = 1$

4.4 Randomness and complexity [7]

Kolmogorov complexity

Kolmogorov complexity of a of binary string *s* (over $\{0,1\}$) is the minimal length of a prefix-free UTM (universal Turing machine) program *x* , which generates *s* :

$K(s) = \min_x \{|x| \mid U(x) = s\}$.

Kolmogorov complexity is equivalent to Boltzmann-Shannon entropy and it defines a **probability** on binary strings (KC-property)

$\sum_{s\in S} 2^{-K(s)} \le 1$ for any enumerable set *S* of binary strings

Randomness of binary strings

There are two equivalent criteria for a binary string $s = s_1 s_2 s_3 ...$ (or equivalently for the corresponding real $r = 0.s_1 s_2 s_3 ...$) to be random.

▪ 1-random

Verbally, a binary string *s* is 1-random, if **Kolmogorov complexity** of its *n*-bit initial parts is **larger than *n*** apart from a positive constant integer *k*

$K(s_1 s_2 ... s_n) > n - k \quad \forall n \ge 1$

▪ Martin-Löf negative

The Martin-Löf criterion tests a binary string *s* for non-randomness, basically it tests, whether s has a **sequence of initial parts** $(u_1, ... u_k, ...)$ so that the length of the **k-th part** u_k **grows faster** than *k* ; if it does, it is ML-positive (non-random)

s ML-positive= $s = (u_1, ... u_k, ...), |(u_1, ... u_k)| > k \quad \forall k \ge k_0$

subdivided in substrings u_k , so that some measure μ of its *n*-bit-long initial parts with KC-property decreases faster than 2^{-n} : if it does, *s* is non-random.

Example: the fast periodic string $s = s_0\,101101...$ with a prefix s_0 and the period 101 is ML-positive and non-random:

$(s_k) = (s_0, s_0 101, s_0 101101, ...)$ and length $|s_k| = |s_0| + 3(k-1) > k, \quad k \ge \frac{3 - |s_0|}{2}$

5 CA in thermodynamics and hydrodynamics

Gasses and fluids consist of molecules and on microscopic scale the dynamics is determined by collision laws (ideal gas) and van-der-Waals-interaction (fluid, real gas). On macroscopic scale, the behavior is determined by

few macroscopic variables: volume, particle number, pressure, temperature, entropy, and by aggregate laws: ideal gas equation-of-state (eos), van-der-Waals eos, Navier-Stokes-equations.
The microscopic laws are reversible (time-symmetric) and obey conservation laws (energy, momentum, particle number).
A closed system reaches in a short time maximum entropy (i.e. disorder), its dynamics is random (thermodynamic variables are random variables , which obey the Boltzmann energy probability distribution), and ergodic (the distribution of occupied states in the state space becomes uniform) because of particle number conservation.
CA can model random aggregate behavior, but its rules must fulfill the criteria

- Reversibility

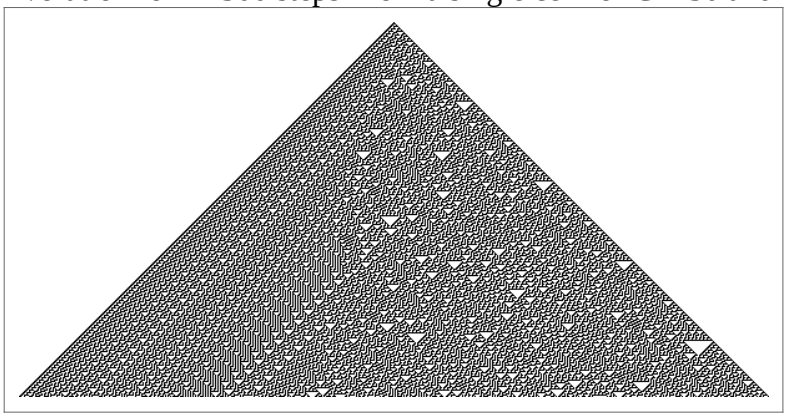
CA maps for step *t* , the preceding string s_t into the output (succeeding) string s_{t+1} : $CA(s_t) = s_{t+1}$
We consider rules, which map a block of *b* cells into another block of *b* cells (blocked CA with *k* colors and block-size *b*).
For reversibility, this mapping must be one-to-one, i.e. it is a permutation of the block pattern..

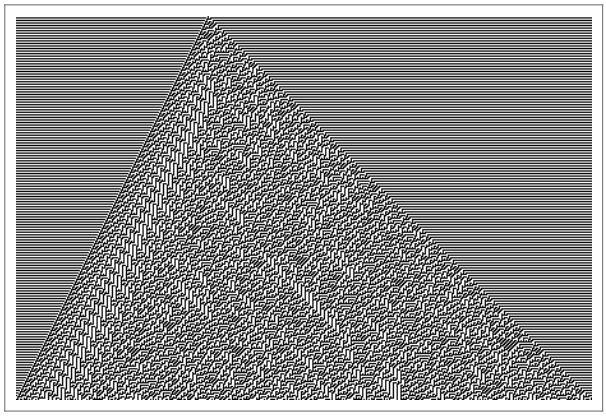
- Conservation laws

Conservation of particle number: the CA conserved variable is $N_{CA} = \nu_0 N_0 + \nu_1 N_1$

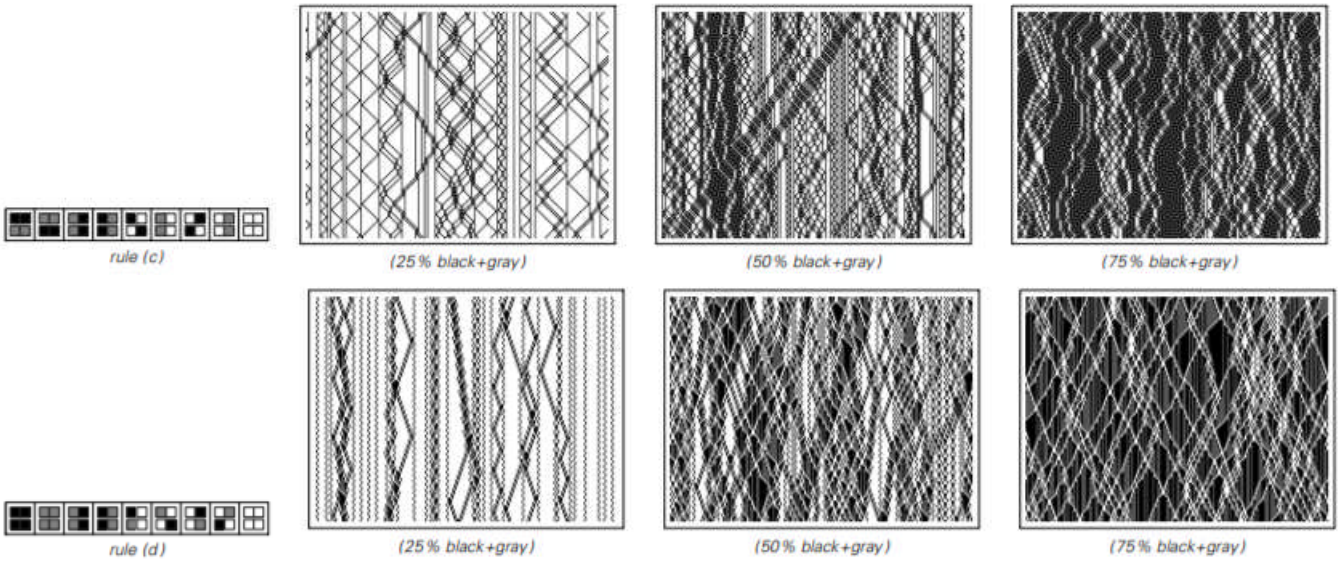
- Instability

In order to generate (almost) random patterns, CA must be unstable with respect to arbitrarily small changes in initial conditions: Lyapunov coefficient $\lambda > 0$.
Examples: CA random generators, rule 30, rule 45
Here the generated strings are random according to all common randomness tests.
Evolution for T=300 steps from a single cell for CAr30 and CAr45



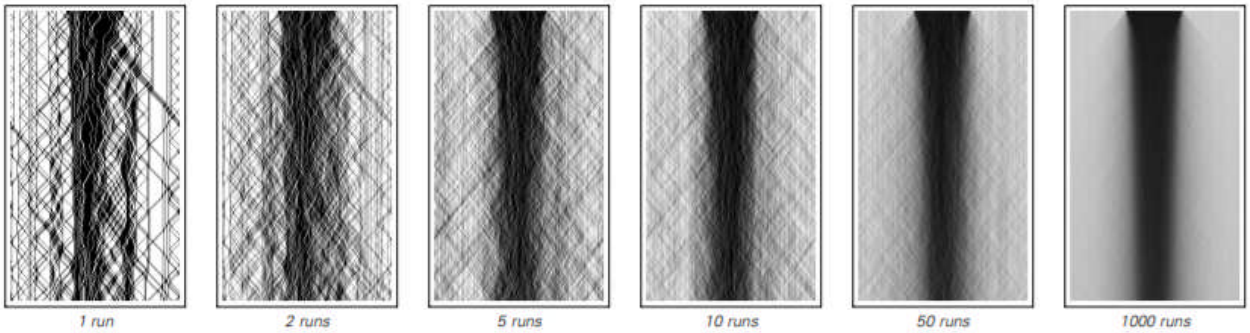


5.1 CA and diffusion [1]
It turns out that for modelling diffusion on a 1-dimensional CA, we need a blocked CA with b=2 and at least k=3 colors (i.e. {0=white,1=gray,2=black}).
The appropriate CA are reversible and conserve the number of gray+black cells.

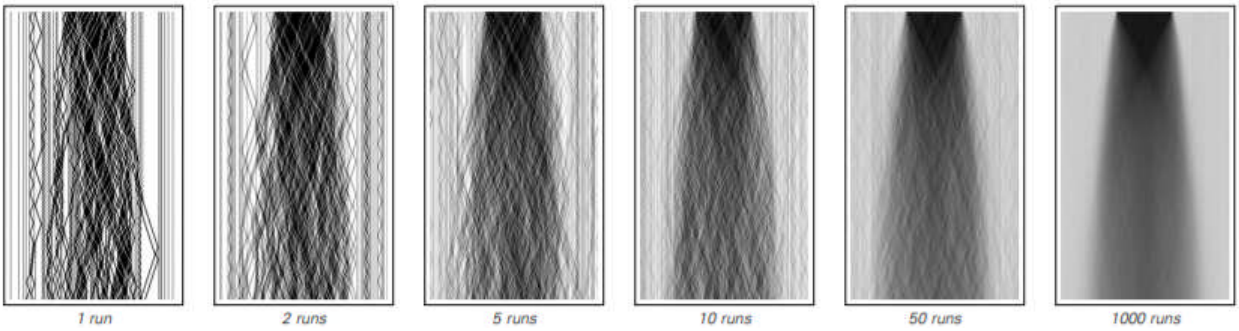


Block cellular automata (rule c, rule d) with three possible colors which conserve the combined number of black and gray cells, with different percentage of gray and black (the rule is shown on the left). The random behavior is especially strong for approximately equal numbers of the 3 colors.

The two CA(rule_c) and CA(rule_d) show indeed diffusion-like behavior, when averaged over slightly differing but similar initial condition with constant density, which mimics fluid entering through a slit.



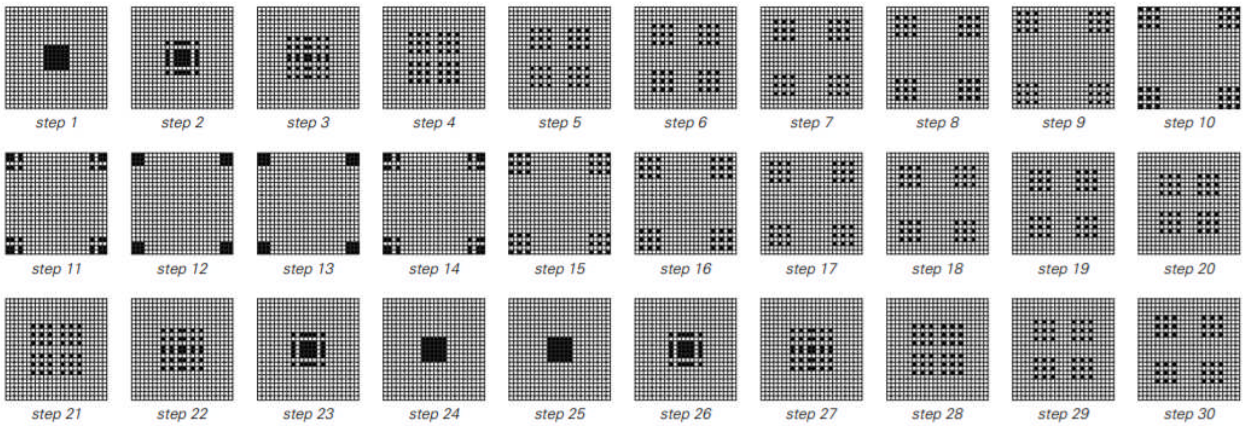
CA(rule_c) averaged over n=1,2,...1000 runs



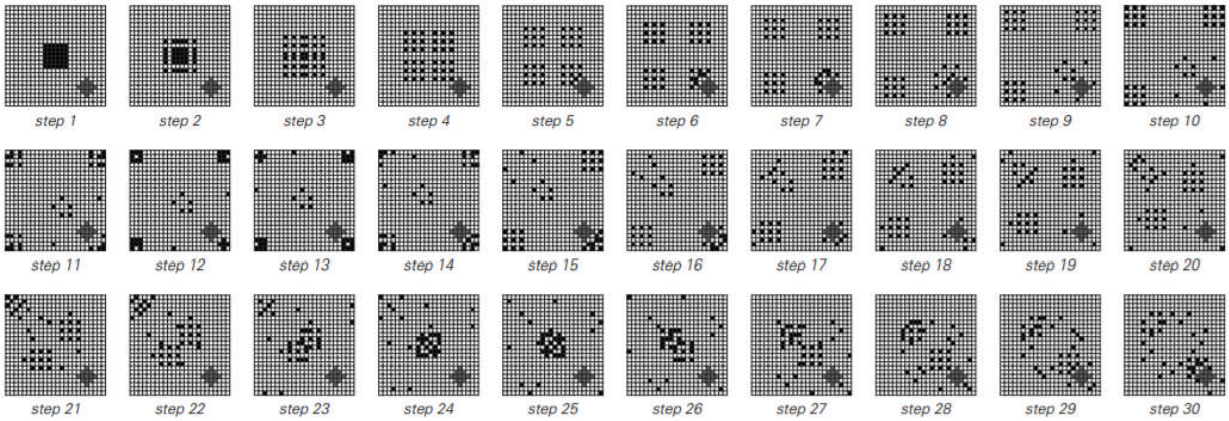
CA(rule_d) averaged over n=1,2,...1000 runs

5.2 CA and ideal gas [1]

There are 2-dimensional CA, which can simulate the behavior of the 2-dimensional ideal gas.
The one shown here, called CA(Lorentz), in fact simulates the behavior of particles moving and colliding in a square box, called Lorentz gas. CA(Lorentz) is a 2x2-blocked 2-dimensional CA, is reversible and conserves the number of particles in a collision.
Lorentz gas becomes chaotic and ergodic, if there is an obstacle in the box, which clearly happens also with CA(Lorentz), as shown below. The analogue case with elliptic boundary is called Yakov-Sinai gas and is always chaotic and ergodic.

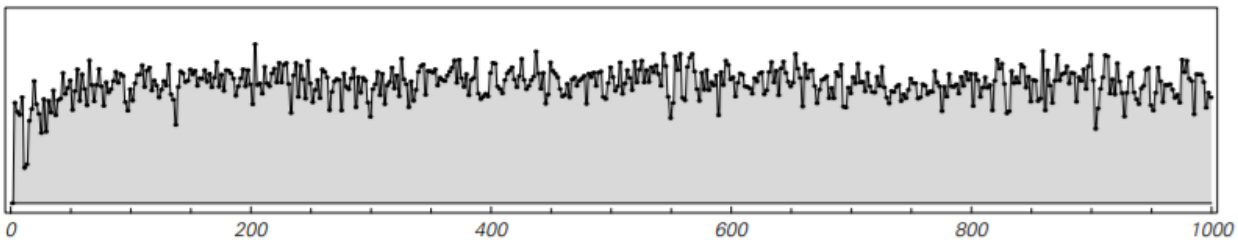


Behavior of CA(Lorentz) without obstacle: the configuration is cyclically expanding and colliding, from the initial state at top-left.



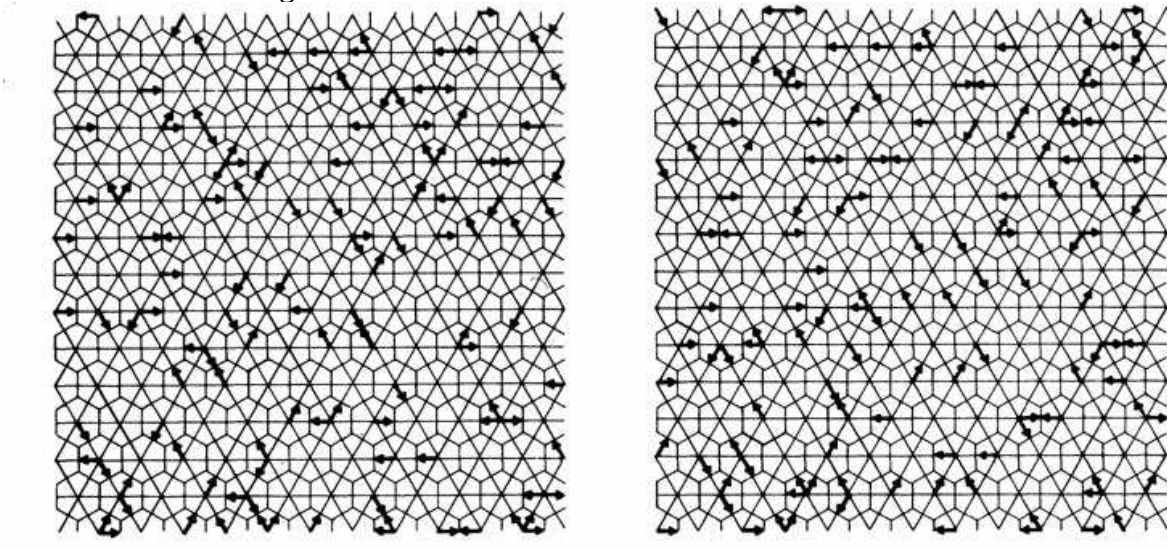
Behavior of CA(Lorentz) with square obstacle at the right-bottom corner, and with the same initial state. The configuration is chaotic and converges to a uniform distribution.

CA(Lorentz) also shows entropic behavior: its coarse-grained increases rapidly at first and stays constant apart from statistical fluctuation. The coarse-grained entropy $S = -\sum_i p_i Log\ p_i$ is calculated on 6x6 patches from the corresponding probabilities p_i



5.3 CA modeling a fluid [2]

The CA considered here, called CA(hex_fluid), is a 2-dimensional CA on the regular hexagonal lattice. Each cell is connected to its 6 neighbors by unit vectors $\vec{e}_1 ... \vec{e}_6$, where $\vec{e}_k = (\mathrm{Cos}(\pi k/3), \mathrm{Sin}(\pi k/3))$. A particle has velocity=momentum given by one of the vectors $\vec{e}_k$, and move accordingly in every step. The collision rules are reversible and are chosen to conserve the number and total momentum of the particles. Two successive configurations are shown below.



The configuration at time *t* and point vector $\vec{x}$ is described by the probability distribution function $f_a(\vec{x},t)$ for particle *a*.

The changes in f_a are small than $\delta f_a(\vec{x},t) << 1/\delta$, where $\delta = \delta_x = \delta_t$,and we get the approximative

transport equation $\partial_t f_a(\vec{x},t)+\vec{e}_a\nabla f_a(\vec{x},t)=\Omega_a(f_a(\vec{x},t))$, where $\Omega_a(f_a(\vec{x},t))$ is the collision term, and the left side is the collision-free Boltzmann transport equation.

We have conservation laws:

$\sum_a f_a = n$ for total particle density

$\sum_a \vec{e}_a f_a = n\vec{u}$ for total momentum density

which yields finally

$$\sum_a (\partial_t + \vec{e}_a\cdot\nabla)\vec{e}_a f_a + \frac{1}{2}\delta\sum_a\left(\vec{e}_a\cdot\nabla\partial_t + (\vec{e}_a\cdot\nabla)^2\right)\vec{e}_a f_a = 0$$

In equilibrium, $f_a(\vec{x},t)$ should depend only on the macroscopic velocity distribution $\vec{u}(\vec{x},t)$ and the particle density distribution $n(\vec{x},t)$.

We use the Chapman-Enskog expansion

$$f_a = f\left(1+c^{(1)}\vec{e}_a\cdot\vec{u}+c^{(2)}\left((\vec{e}_a\cdot\vec{u})^2-\frac{1}{2}|\vec{u}|^2\right)+c_\nabla^{(2)}\left((\vec{e}_a\cdot\nabla)(\vec{e}_a\cdot\vec{u})-\frac{1}{2}\nabla\cdot\vec{u}\right)+...\right)$$

and we get $f = n/6$ $c^{(1)}=2$ pressure $p = n/2$

We get the final CA macroscopic equation

$$\partial_t(n\vec{u})+\frac{1}{4}nc^{(2)}\left((\vec{u}\cdot\nabla)\vec{u}+\vec{u}(\vec{u}\cdot\nabla)+\left(\vec{u}(\nabla\cdot\vec{u})-\frac{1}{2}\nabla|\vec{u}|^2-\frac{1}{2}|\vec{u}|^2\nabla\right)\right)=$$

$$-\frac{1}{2}\nabla n-\frac{1}{8}nc_\nabla^{(2)}\left(\nabla^2\vec{u}+2\nabla\cdot\vec{u}\right)+\frac{1}{8}(\nabla\cdot\vec{u})\nabla nc_\nabla^{(2)}$$

The Navier-Stokes equations are given in the form (d=dimension, here d=2)

$$\partial_t(n\vec{u})+\frac{1}{4}\mu n(\vec{u}\cdot\nabla)\vec{u} = -\nabla p+\eta\nabla^2\vec{u}+\left(\zeta+\frac{\eta}{d}\right)\nabla(\nabla\cdot\vec{u})$$

where p=pressure, η =shear viscosity, ζ =bulk viscosity

By comparison with the CA macroscopic equation we get

$$\zeta = 0 \quad \eta = -\frac{1}{4}nc_\nabla^{(2)} \quad \mu = \frac{1}{4}c^{(2)} \quad p = n/2$$

Part 2 Applications of Cellular Automata

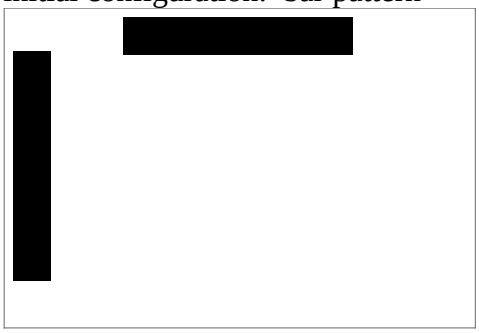
1 Characterization of CA

We need criteria for effective description of the string set generated by a specific CA (its output). It is difficult to describe the output only on the basis of the CA rule and the initial input (the start configuration).
In fact, for class4 CA, it is equivalent to specifying the output of a Turing machine, which is in general non-computable (specifically for the halting problem, which is not algorithmically decidable).
So we have to analyze the output of each step with a few parameters in order to handle the enormous amount of data arising there. In case of 2-dimensional CA, it can be done with means of image analysis and entropy calculation.

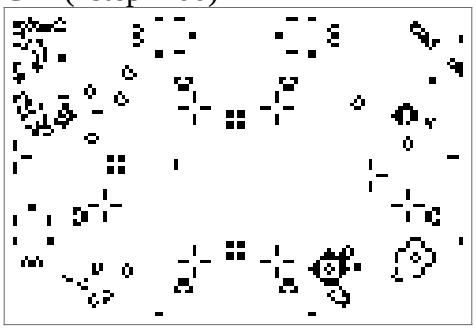
1.1 Morphological component analysis

The image analysis must be able to recognize the morphological components of the image independent of their position and rotation, and it should be also tolerant to small differences in size, in other words, it should detect "similar structure" and return a quantitative result for it.
Wolfram's built-in image analysis in Mathematica is a powerful and precise tool for recognition and quantitative description of morphological components, in fact, there is a function called which does exactly this.
m=MorphologicalComponents[CAx];
sets up a list of recognized structural components and
compsl=ComponentMeasurements[{m,img},{"Count","Length","Width"}];
inserts the parameters *Count*=number of pixels, geometrical *Length* (maximal diameter), geometrical *Width* (orthogonal diameter)- into this list.
Example: Game-of-Life (rule cs({Sn=3},{Sn=2,*c*=1})=1) starting with a bar-pattern , at nstep=100

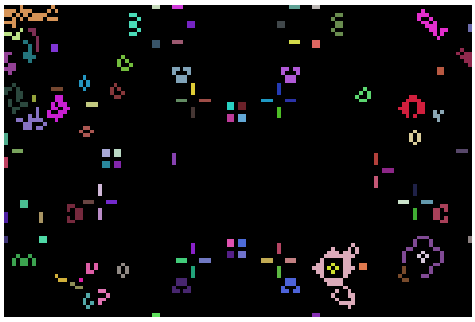
initial configuration: bar pattern



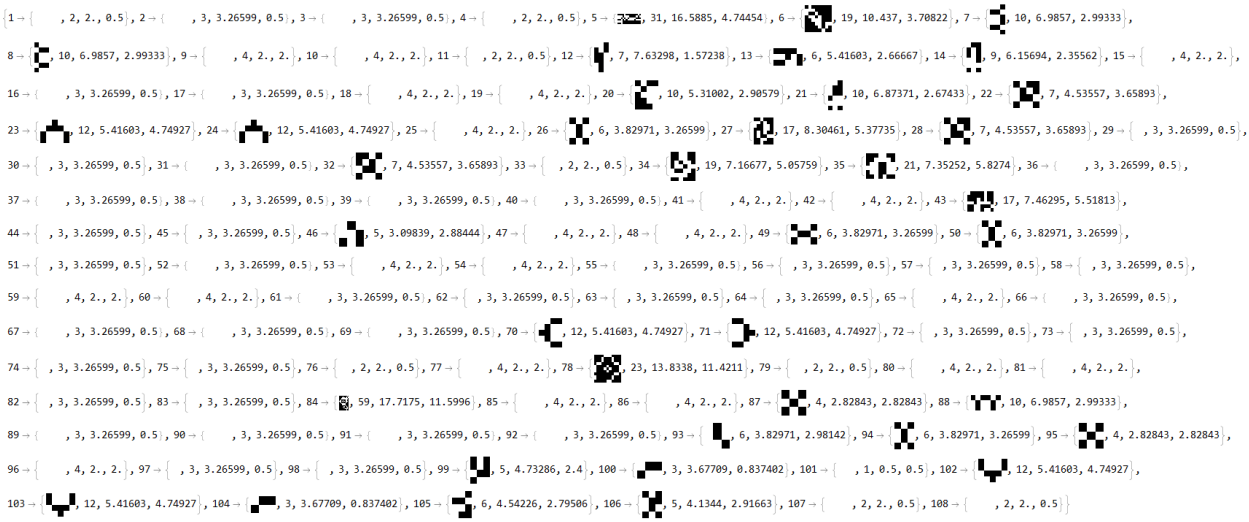
CA1(nstep=100)



recognized components distinguished by colors



list of components with image, number pixels, length, width



1.2 Entropy

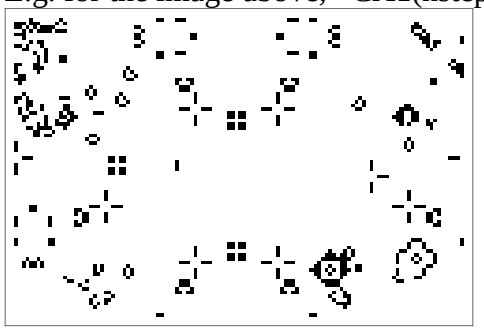
Another important parameter of the output is the entropy, as a measure of complexity .
Originally, the Shannon entropy for $N(n)$ of sequences of length *n*, is

$$H(n) = -\sum_n p_n \log_2 p_n = -\sum_n \frac{N(n)}{2^n} \log_2\left(\frac{N(n)}{2^n}\right)$$

A good approximation for *H(n)* is the value calculated for the morphological components

$$H_{mc}(n) = -\sum_{n=L(mc)} \frac{N(n)}{2^n} \log_2\left(\frac{N(n)}{2^n}\right)$$

E.g. for the image above, CA1(nstep=100)



we get *H(n)= 27.43*

The output of a modified Game-of-life (rule cs({Sn=2},{Sn=3,*c*=1})=1) at nstep=100 is : CA2(nstep=100)

has the entropy of *H(n)= 114.65*

The second image is much more complex (i.e. “chaotic”) as the first, and its entropy is much higher (3x as large) than with the first image, as expected.

2 Goals for CA evolution

Evolution in its basic meaning is of course the Darwinian biological evolution on Earth.
In the general meaning it can be described as self-modification of a functional system with input and output in order to adapt its output function to a prescribed pattern (=goal).

In the biological context, the system is the genotype of a species equipped with input sensors and output actuators, the functionality is the brain and the nervous system with its inherited and learned behavior, the goal is survival, and the self-modification is the genetic reproduction and mutation mechanism.

In the context of a learning and evolving robot driving an automatic car through the traffic, the system is an artificial-intelligence-machine (i.e. CPU+neural network (NN) equipped with sensors and actuators controlling the car), the goal is to bring the car from point A to point B in an acceptable time obeying the traffic rules and avoiding collisions and accidents, the self-modification is the self-adaptation of the NN when learning from new experience by evaluation of degree of success.

In the context of a CA with 2 colors (value=0,1), the input is the initial configuration, the output is the generated configuration (bit-string or binary image) in a certain step, the functionality is the CA rule, the goal is a chosen pattern (bit-string or binary image), and the self-modification is an algorithm which corrects the parameters (the step number, the rule and the initial input) guided by the deviation of the output from the goal. Here the parameters (the step number, the rule and the initial input) are assumed to be independent of the goal, in general the start values for step number and initial input are random.

2.1 Choice of goal function

-goal= pattern

We can set the goal function equal to the chosen pattern (when interpreted as binary image: with fixed position and rotation), but experience shows [3], that in this case the algorithm will not converge, except when the choose the initial configuration and the rule specifically to fit the goal, which violates the condition of independence.

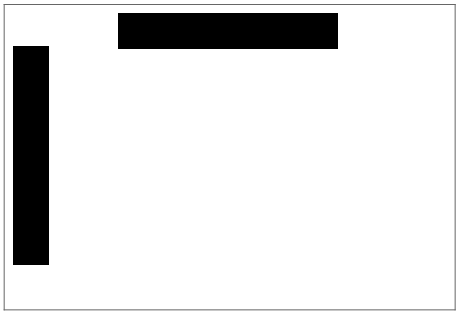
-goal=component parameters of the pattern

A much better alternative is to extract parameters from the chosen pattern by characterization, e.g. from component analysis, and set the goal function equal to these parameters. Here we chose the parameters list(component area, component length, component width) and the entropy to be the goal function.
These parameters are independent of position and rotation, so they describe more the similarity than the identity with a pattern.
With this goal function both the gradient and the genetic algorithm mostly reach a local minimum, as the calculation shows [3].

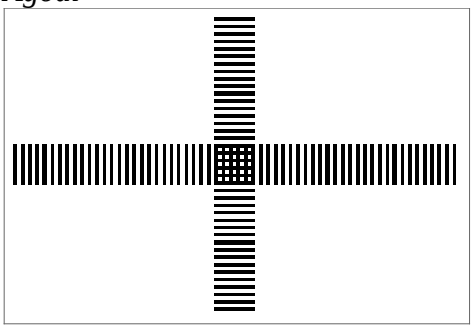
2.2 The actual calculation

In the calculated example [3], we chose different variants of Game-of-Life (as initial rule) *cslist* , with different variants of a two-bars image as initial input *Ain* , and step number *nstart≈20* . The goal pattern (binary image) *Agoal* was chosen to be the “segmented cross” .

initial *Ain*=



Agoal=



3 Evolution with gradient algorithm

The gradient algorithm with its variants Levenberg-Marquardt and Conjugated-Gradients is presumably the best-known solution method for optimization problems. This method is applicable only for continuous goal functions, so one has to discretize the result for the rule and the initial input.

3.1 Algorithm flowchart

The parameters are (*nstart, cslist, Ain*) = (step number, rule, initial input), the goal function is the distance of the component parameter vectors and the entropy difference between the current CA (*CAx*) and the goal pattern (*Agoal*) :

dev(CAx , Agoal)=dist(param(CAx) , param(Agoal)) + entropy(CAx)-entropy(Agoal)

iteration 0 *it=0,* initialization of running parameters (*nstartx, cslistx, Ainx*) = (*nstart, cslist, Ain*) , *weightx = value(cslistx) , Aweightx = value(Ainx) , nweightx = value(nstartx)* , iteration limit *Nit*

iteration : *it=it+1 ; until it=Nit*

gradient loop *cslistx*: local variable *cslisty* , *weighty* , init *cslisty = cslistx*
invert *cslisty[i]=1- cslistx[i]*
calculate step sign *if cslistx[i] =0, sign=+1; else sign=-1*
determine local CA *CAy=CA(nstartx, cslisty, Ainx)*
calculate deviation *weighty[i]= sign dev(CAy , Agoal)*
update : *weightx [i] = weightx [i] + weighty [i]*
discretize and update: *if weightx [i] ≥0.5, cslistx[i] =1; else cslistx[i] =0*
endloop

gradient loop *Ainx*: local variable *Ainy , Aweighty* , init *Ainy= Ainx*
invert *Ainy[i]=1- Ainx[i]*
calculate step sign *if Ainx[i] =0, sign=+1; else sign=-1*
determine local CA *CAy=CA(nstartx, cslistx, Ainy)*
calculate deviation *Aweighty[i]= sign dev(CAy , Agoal)*
update : *Aweightx [i] = Aweightx [i] + Aweighty [i]*
discretize and update : *if Aweightx [i] ≥0.5, Ainx[i] =1; else Ainx[i] =0*
endloop

correction *nstartx*
increase local *nstarty =nstarty+1*
determine local CA *CAy=CA(nstarty, cslistx, Ainx)*
calculate deviation *Aweighty[i]= sign dev(CAy , Agoal)*
update *nstartx =nstarty+nweighty*

determine new CA : *CA=CA(nstartx, cslistx, Ainx)*
calculate deviation *devx= dev(CAx , Agoal)*

iterate: *goto iteration*

3.2 Results

Starting with dev=490, entropy1=entropy(CAx)=3.43 , the iteration gets stuck in the step it=3, with dev=228, entropy1=0.437, entropy2= entropy(Agoal)=0.472, nstartx=11

"time="0.`" it="1" dev="490.3237632018637`" nstartx="10" entropy1="3.4350686073304377`" entropy2="0.47265625`

"time="727.7599999999948`" it="2" dev="228.56227892390172`" nstartx="11" entropy1="0.4375`" entropy2="0.47265625`

"time="1459.9669999999896`" it="3" dev="228.56227892390172`" nstartx="11" entropy1="0.4375`" entropy2="0.47265625`

With a random modification *nstartx=nstartx±1* comes out from the local minimum, but does not improve within 8 iterations

"time="0.`" it="1" dev="228.56227892390172`" nstartx="11" entropy1="0.4375`" entropy2="0.47265625`

"time="723.7669999999925`" it="2" dev="1911.9062607836645`" nstartx="12" entropy1="16.37500000032287`" entropy2="0.47265625`

"time="1453.508999999991`" it="3" dev="782.4517931492131`" nstartx="13" entropy1="6.0009765625`" entropy2="0.47265625`

"time="2184.7939999999944`" it="4" dev="1019.6107326445265`" nstartx="14" entropy1="6.250000000278419`" entropy2="0.47265625`

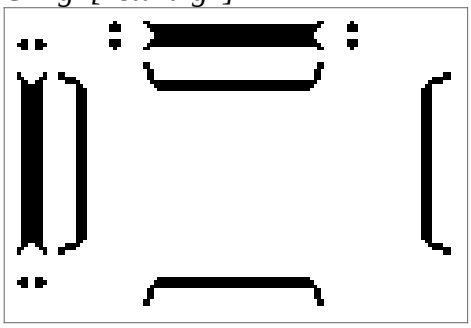
"time="2915.7219999999943`" it="5" dev="549.5326586706525`" nstartx="15" entropy1="2.97265625`" entropy2="0.47265625`

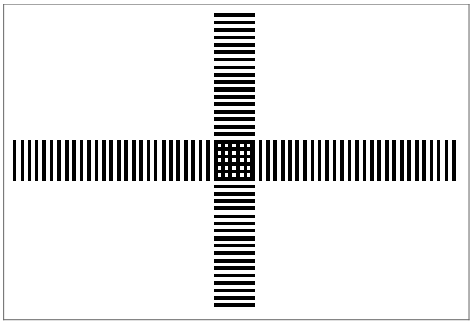
"time="3646.180999999997`" it="6" dev="990.8163029190582`" nstartx="14" entropy1="5.87500000014802`" entropy2="0.47265625`

"time="4378.231`" it="7" dev="549.5326586706525`" nstartx="15" entropy1="2.97265625`" entropy2="0.47265625`

"time="5108.627999999997`" it="8" dev="523.7219066902102`" nstartx="15" entropy1="2.6601805686950684`" entropy2="0.47265625`

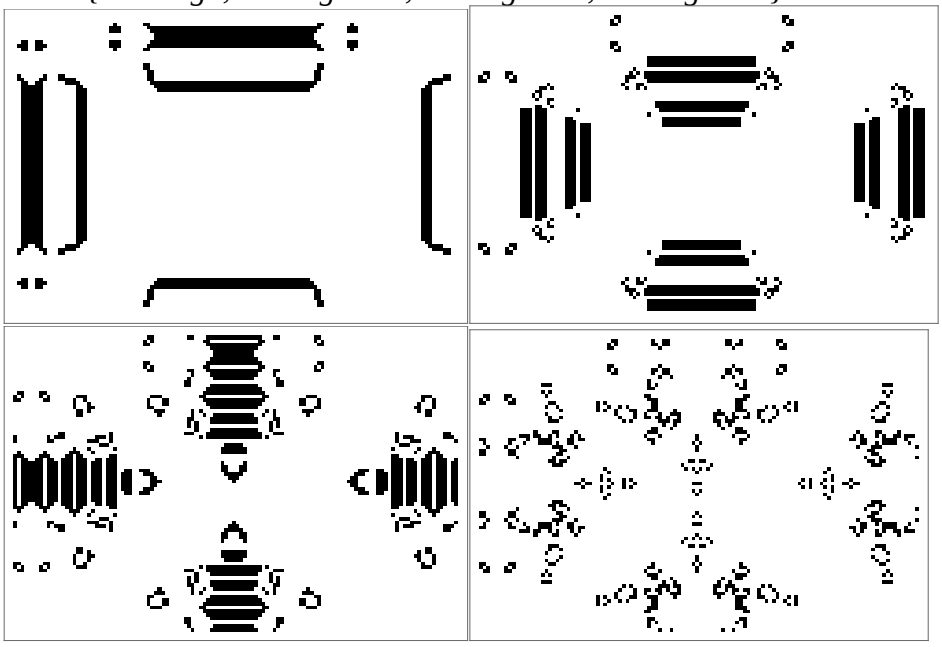
So the best actual result is the local minimum at dev=228 . The corresponding CA=*CAxg2* has the output *CAxg2[nstartxg2]=*





where *Agoal=*

The evolution in indices from the list *iCAx={nstartxg2,nstartxg2+10,nstartxg2+20,nstartxg2+30}* is



4 Evolution with genetic algorithm

Genetic algorithms are optimization methods, which are analogous to the biological cell reproduction in terrestrial life evolution. Two candidate parameter sets of *n* parameters are "crossed", i.e. parameter values are extracted from both sets, in general randomly, and form a new set of *n* parameters . The new and old candidate parameter sets are then evaluated, and the best are chosen for the next crossing event [4] [5].
The genetic algorithm starts with a fixed number *nCAx* of initial CA, (here *nCAx=10*) , where the first *nCAxm* (here *nCAxm*=5) are the "best" (minimal deviation *dev*) . In every iteration step these best CA are selected for reproduction, i.e. they are crossed at random among themselves and subjected to mutation with a certain percentage *prand* (here *prand=0.2*) and generate the remaining *nCAx- nCAxm* CA. After reproduction and mutation, the deviations of the CA are calculated, and they are reordered depending on the deviation, so the first *nCAxm* are again the best. Then the next iteration step starts.

4.1 Algorithm flowchart

The actual CA vector is *CAa=(CA[i] | i=1,...,nCAx)* .
The parameters are (*nstarta, cslista, Aina*) = (step number vector, rule vector, initial input vector), the goal function is the distance of the component parameter vectors and the entropy difference between the current CA (*CAa[k]*) and the goal pattern (*Agoal*) :
deva=(dev(CAa[k] , Agoal)=dist(param(CAa[k]) , param(Agoal)) + entropy(CAa[k])-entropy(Agoal) | i=1,...,nCAx)
Here the best first *nCAxm* rules *cslista[i]* are initialized with variants of Game-of –Life, and the first *nCAxm* initial inputs *Aina* is initialized with variants of the two-bars pattern.

iteration 0 *it=0*, initialization of running parameters (*nstarta, cslista, Aina*) ,*deva* , iteration limit *Nit*

iteration : *it=it+1 ; until it=Nit*

loop reproduction it1=1,.., *nCAx-nCAxm*
select random index pairs for reproduction: *{i1,i2}=random[1... nCAxm]*
update *cslistx*:
cross i1, i2 :*cslistx=cross(cslista[i1], cslista[i2])*
mutate i1, i2 : *irand=sample(1...L(cslistx), count=prand*L(cslistx)) ; cslistx[irand]=1- cslistx[irand];*
cslista[it1]=cslistx;
update *Ainx*:
cross i1, i2 : *Ainx =cross(Aina[i1], Aina[i2])*
Aina[it1]= Ainx;
update *nstartx* : *nstartx=(nstarta[i1]+ nstarta[i2])/2*;
randomize: *nstartx= nstartx+random(-2*prand*nstartx, 2*prand*nstartx);*
nstarta[it1]= nstartx;

set CA: *CAa[nCAxm+it1]=CA(nstartx, cslistx, Ainx)*;

endloop;

calculate deviation: *deva=(dev(CAa[k] , Agoal)=dist(param(CAa[k]) , param(Agoal)) + entropy(CAa[k])-entropy(Agoal) | i=1,...,nCAx)*
order ascending :index *ideva=order(index(deva), ascending dev);*
reorder CA: *CAa=CAa[ideva];*
cslista=cslista[ideva];
Aina=Aina[ideva];
nstarta= nstarta [ideva];

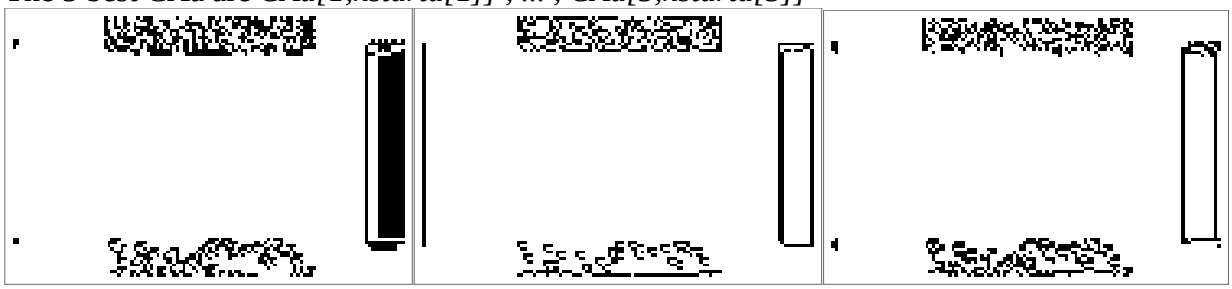
iterate: *goto iteration*

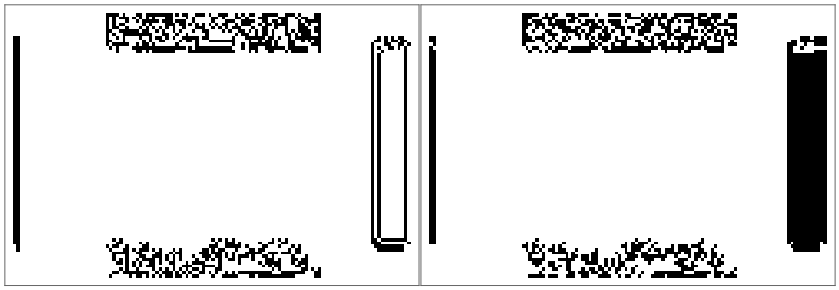
4.2 Results genetic algorithm with mutation

The mutation percentage for the rule (*cslistx*) is prand=0.2 .
The algorithm starts with best dev=490.3 and reaches 419.7 after 500 iterations

```
time=2.932 it=1 deva={490.324, 982.598, 982.598, 982.598, 2257.88, 2267.6, 2702.73, 3600.02, 3974.79, 4579.97} nstarta={10, 10, 10, 10, 11, 6, 10, 8, 10, 8}
time=1820.63 it=500 deva={419.746, 426.353, 432.37, 433.347, 451.443, 555.971, 868.017, 978.425, 1548.03, 1816.19} nstarta={3, 2, 3, 3, 3, 3, 2, 2, 2, 4}
```

Apparently, the alorithm fails to find a local minimum, because of mutation (see below).
The 5 best CAa are *CAa[1,nstarta[1]] , ... , CAa[5,nstarta[5]]*

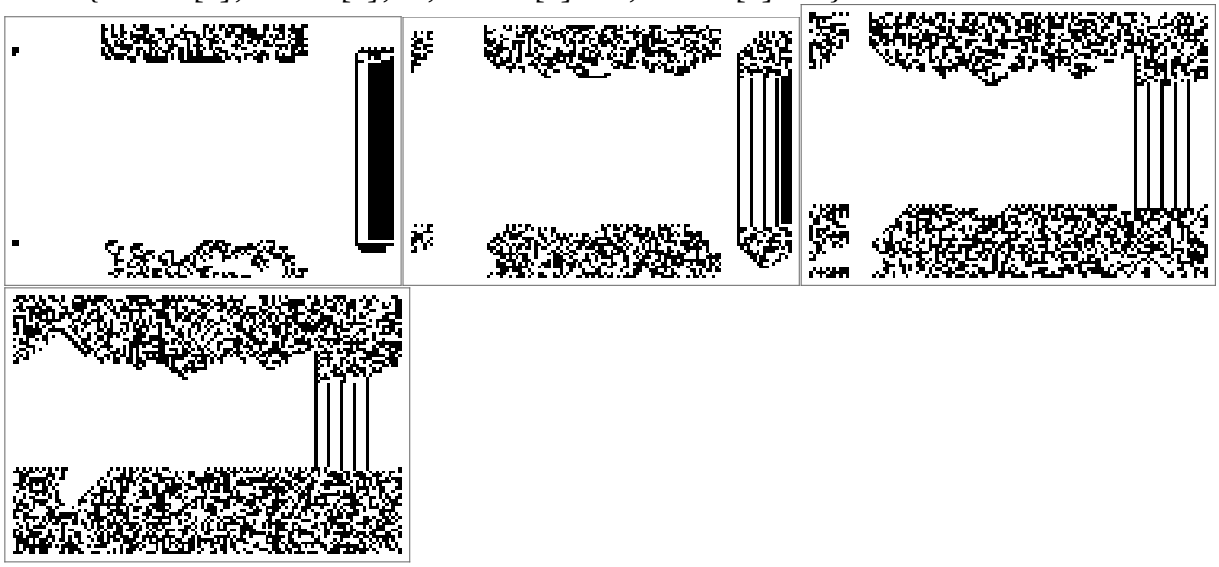




with deviation and entropy

```
ix=1 dev=419.746 entropy=2.53662 entropygoal=0.472656
ix=2 dev=426.353 entropy=1.60651 entropygoal=0.472656
ix=3 dev=432.37 entropy=2.49438 entropygoal=0.472656
ix=4 dev=433.347 entropy=2.5 entropygoal=0.472656
ix=5 dev=451.443 entropy=2.46875 entropygoal=0.472656
```

The evolution of *CAa[1]* in indices from the list
iCAx={nstarta[1], nstarta[1],+5, nstarta[1]+10, nstarta[1]+15} is

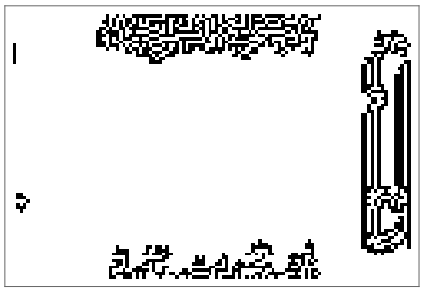


4.3 Results genetic algorithm without mutation

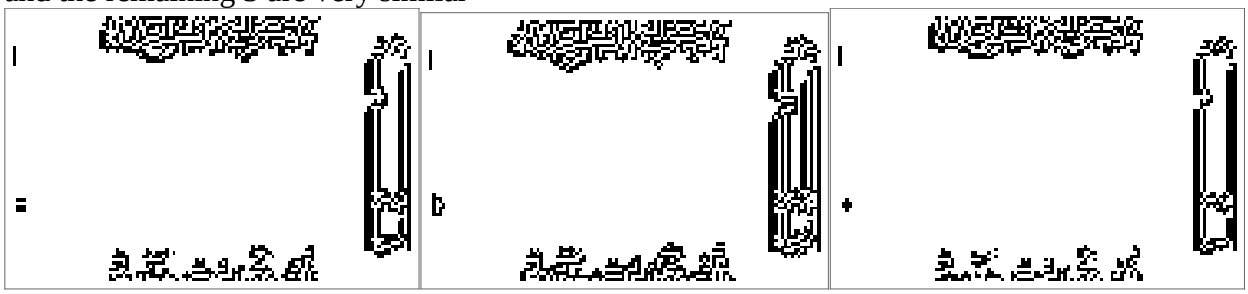
Here the mutation for the rule (*cslistx*) is skipped . Without mutation, the result is much better: the algorithm starts with dev=490.3 as before and reaches a local minimum dev=306.7 , after 1500 steps, 7 from 10 CA are identical best.
The algorithm starts with best dev=490.3 and reaches 306.7 after 1500 iterations

```
time=3.026 it=1 deva={490.324, 982.598, 982.598, 982.598, 1033.01, 1770.2, 1927.53, 2702.73, 3131.48, 3294.33} nstarta={10, 10, 10, 10, 9, 10, 7, 10, 7, 12}
time=5520.16 it=1500 deva={306.777, 306.777, 306.777, 306.777, 306.777, 306.777, 306.777, 455.054, 479.209, 696.472} nstarta={6, 6, 6, 6, 6, 6, 6, 5, 7, 4}
```

The 7 best CAa are identical *CAa[1,nstarta[1]] , ... , CAa[7,nstarta[7]]*



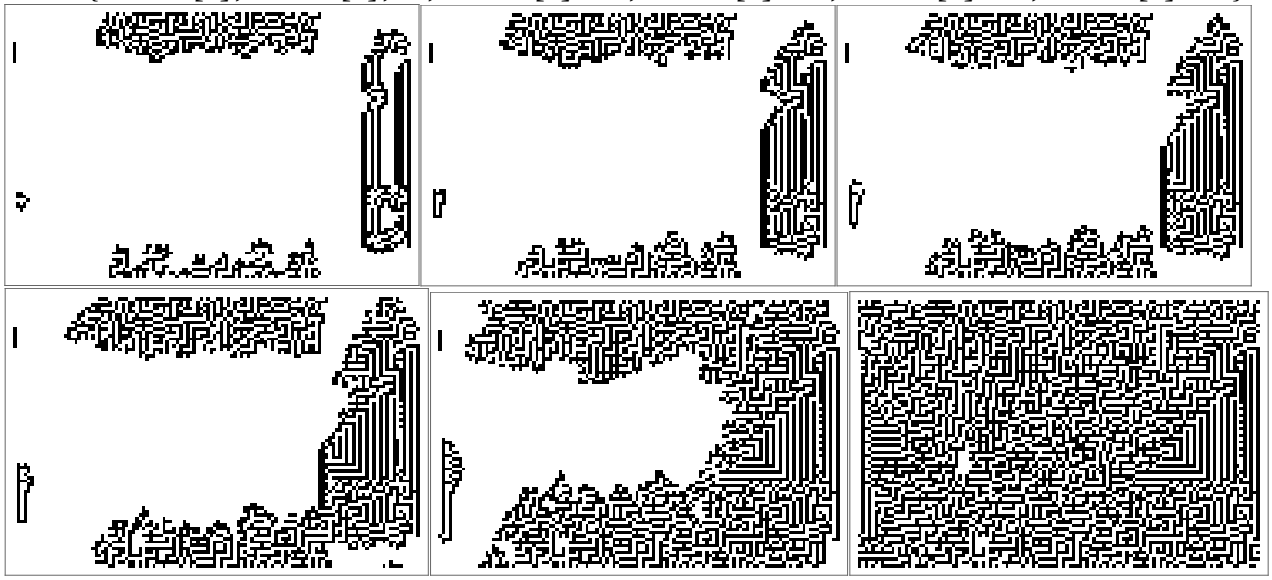
and the remaining 3 are very similar



with deviation and entropy

```
ix=1 dev=306.777 entropy=1.23933 entropygoal=0.472656
ix=2 dev=306.777 entropy=1.23933 entropygoal=0.472656
ix=3 dev=306.777 entropy=1.23933 entropygoal=0.472656
ix=4 dev=306.777 entropy=1.23933 entropygoal=0.472656
ix=5 dev=306.777 entropy=1.23933 entropygoal=0.472656
ix=6 dev=306.777 entropy=1.23933 entropygoal=0.472656
ix=7 dev=306.777 entropy=1.23933 entropygoal=0.472656
ix=8 dev=455.054 entropy=2.71921 entropygoal=0.472656
ix=9 dev=479.209 entropy=2.9375 entropygoal=0.472656
ix=10 dev=696.472 entropy=4.27832 entropygoal=0.472656
```

The evolution of *CAa[1]* in indices from the list
iCAx={nstarta[1], nstarta[1],+5, nstarta[1]+10, nstarta[1]+15, nstarta[1]+35, nstarta[1]+94} is



At nstep=100 , the output is fully “fractal”, entropy= 21.6, and the deviation dev=2376 .

```
dev=2376.08 entropy=21.6968 entropygoal=0.472656
```

Part 3 Multiway systems

1. Basics of multiway systems

1.1 Basics of Multiway Cellular Automata (MCA)

Multiway state automata are a generalization of state automata [8] , [9], insofar as the *unique transition rule* from one state to another is extended *to several rules executed in parallel*.

When applied to Cellular Automata (CA), it means that the transition graph of a *Multiway Cellular Automaton* (MCA) has several edges leading from and to a node, whereas the transition graph of a CA has at most one edge leading from a node and at most one leading to a node.

In [8] MCA graphs are called *substitution systems*.

Any CA has two basic components: initial configuration=bit-string and transition rule, which transforms the one string into another string. E.g. for 1-dim CA: configuration= 3-tuple (c,l, r), new value= c', rule = (($(c_1,l_1, r_1),c_1'$),,..) , where ($(c_1,l_1, r_1),c_1'$) represents the transition $(c_1,l_1, r_1)\rightarrow c_1'=t(c_1)$, the graph is a sequential graph.

For a MCA, we have several rules, e.g. . for 1-dim MCA: configuration= 3-tuple (c,l, r), new value= c', rule1 = (($(c_1,l_1, r_1),c_1'$),,..) , rule = (($(c_1,l_1, r_1),c_2'$),,..) , transition1 $(c_1,l_1, r_1)\rightarrow c_1'= t_1(c_1)$,
transition2 $(c_1,l_1, r_1)\rightarrow c_2'= t_2(c_1)$. The transition graph is now a tree graph.

Example [8]

simple CA

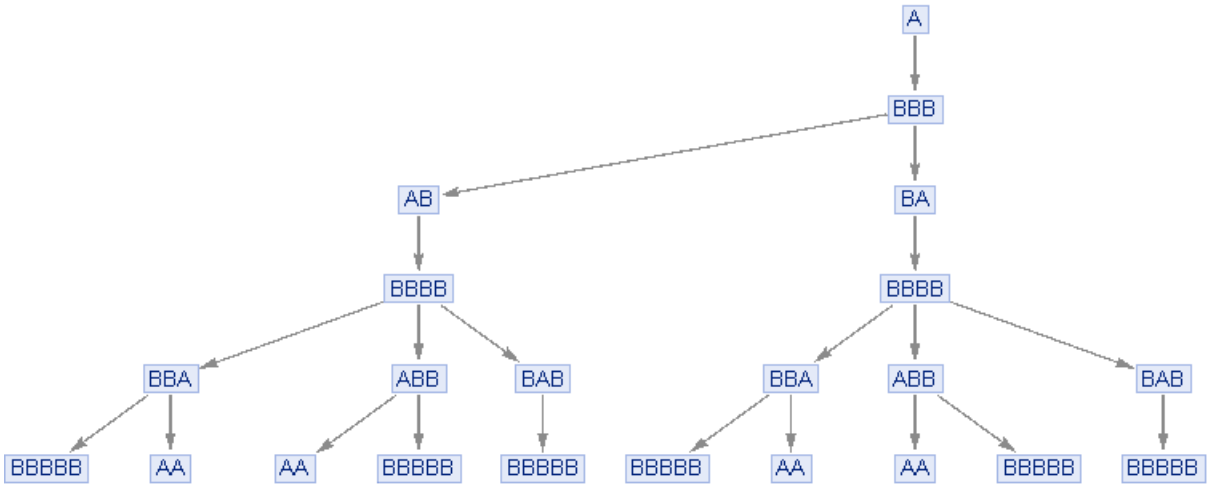
initial configuration string s={A} , rule $A \rightarrow BBB$

transition graph

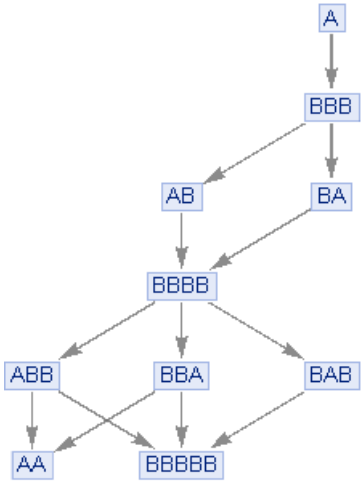
extended rule MCA

initial configuration string s={A} , rule $\{A \rightarrow BBB, BB \rightarrow A\}$

evolution graph



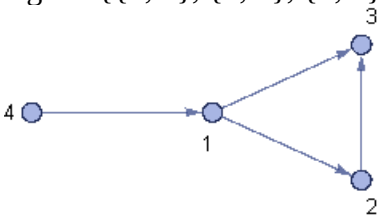
state graph (i.e. identical strings merged into one state)



1.2 Basics of Hypergraph Transition Automata (HTA)

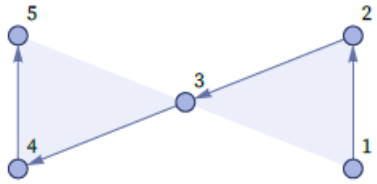
A *directed graph* can be represented by a (binary) *2-relation* $R = \{\{x_{11}, x_{12}\}, \{x_{21}, x_{22}\}, \ldots\}$

e.g. R={{1, 2}, {1, 3}, {2, 3}, {4, 1}} represents the directed graph [8]



This concept can be generalized to *hypergraphs* , which are represented by *multi-relations*,

e.g. the (ternary) 3–relation R={{1, 2, 3}, {3, 4, 5}} represents the directed hypergraph [8]



, here the shaded triangles signify the individual 3–tuples of the relation R .

In analogy to MCA, which transforms strings into strings, a *Hypergraph Transition Automaton* (HTA) transforms hypergraphs into hypergraphs.

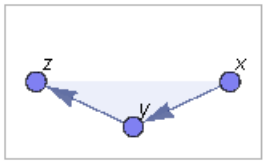
In [8] HTA are called *multiway systems*.

Example [8]

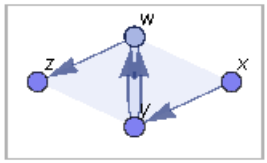
initial config h= {{1,1,1}} rule R= {{x, y, z}} → {{x, y, w}, {y, w, z}},

in hypergraph- representation

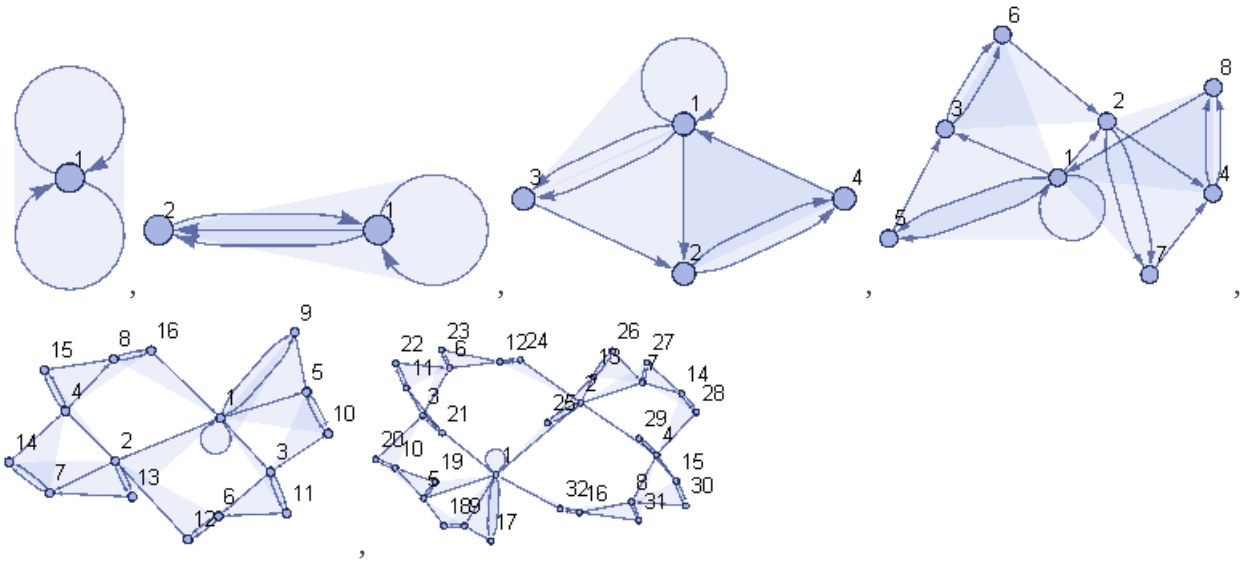
h=



R=



Consecutive application of the rule R yields the following hypergraph outputs



-Full graph and causal (state) graph

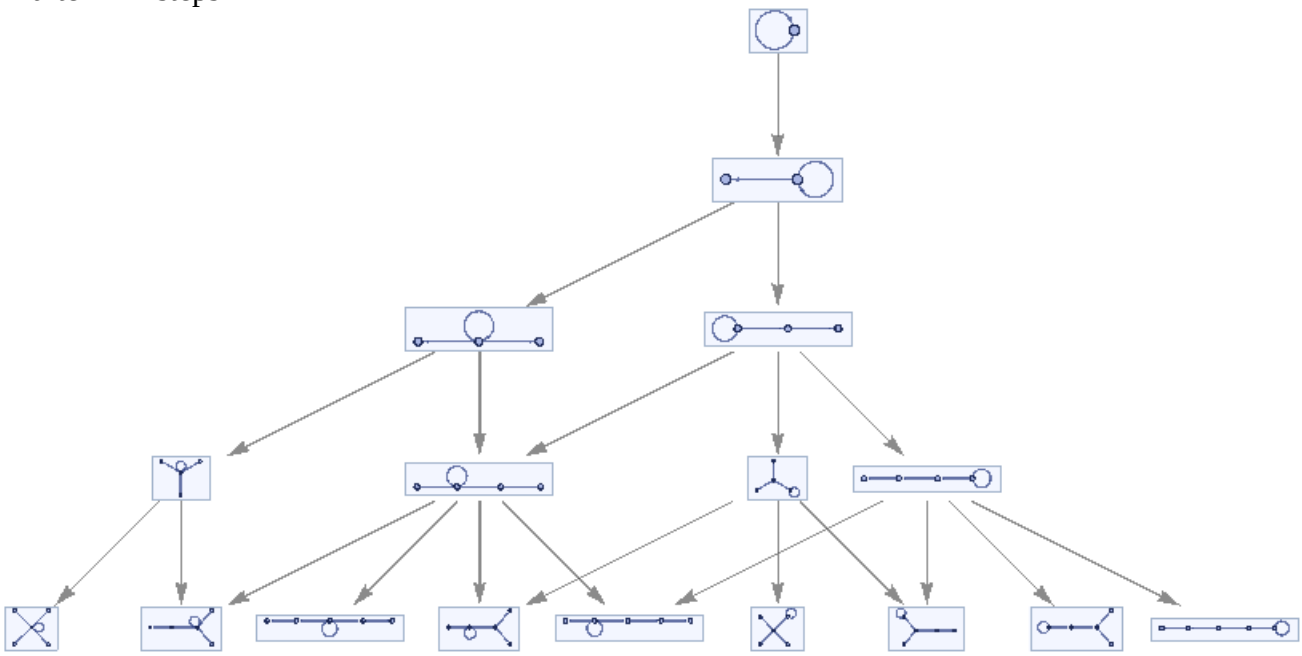
There are basically two graph representation of a HTA: the full (=evolution) graph and the simpler partially merged causal (=state) graph.

Example [11]

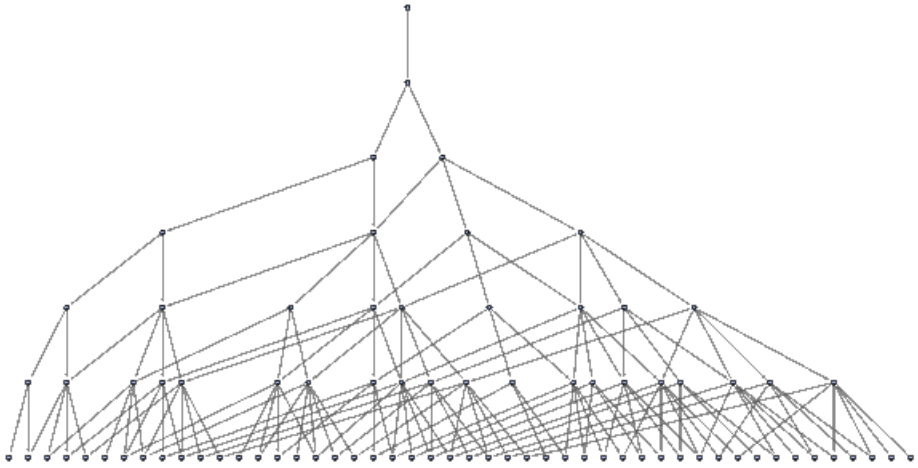
$1_2 \to 2_2$ rule $\{\{1,2\}\} \to \{\{1,2\},\{2,3\}\}$

initial state {{1,1}}

HTA after n=4 steps



the corresponding causal(=state) graph is with n=6



2. Properties of HTA

2.1 HTA and their connections to physics

In the following we examine applications of HTA in physics. There are two properties, which qualify HTA as models of physical events: causal invariance and embedding in physical space-time.

-Causal invariance

First, we define *reflexive transitive closure (rtc)* rules, indicated by the notation $\to *$.
The rule set $S_r = \{r_i, i = 1,...,n\}$ is reflexive transitive closure (rtc),
if it is reflexive (=invertible):
if $r_i = h_{i1} \to h_{i2}$ $r_i \in S_r$ is a rule, then also inverted $r'_i = h_{i2} \to h_{i1}$ is a rule $r'_i \in S_r$,
notation $r_i = h_{i1} \to *h_{i2}$
and transitive (fulfilled automatically by consecutive application):
if $r_i = h_{i1} \to h_{i2}$ and $r'_j = h_{j1} \to h_{j2}$ $r_i \in S_r$, $r'_j \in S_r$, then also $r''_k = r'_j \circ r_i$ is a rule $r''_k \in S_r$
So effectively **rule set S_r is rtc**, **if it contains the inversion of itself** [10].

HTA is called *confluent*, if its rule set is rtc and fulfills the Church-Rosser property:
if $(a \to *b) \in S_r$, $(a \to *c) \in S_r$, then there is a common output $\exists d : (b \to *d) \in S_r$, $(c \to *d) \in S_r$

HTA *h* is called *causal invariant*, if its output under a sequence of transitions is isomorphic (equal under renumeration) under arbitrary sequences.

We have the following theorem (Church-Rosser)
A causal invariant HTA is confluent.

-Embedding of HTA in 4-dimensional Lorentz space-time

Theorem
A causal invariant acyclic binary (2-relation) HTA *h* with output degree *n* can be *embedded in n-dimensional discrete Lorentzian space-time,* where time *t* is the *n*-th transition step directed (vertically) downwards, and space dimensions $x_1,...,x_n$ count the *i*-th output node in the node transition $a_0 \to (a_1,...,a_n)$.
This means that the output nodes of a node lie in different spatial dimensions.

With this embedding, we can use the discrete Minkowski metric $\|(t,\vec{x})\| = |\vec{x}|^2 - t^2$ and define relative velocity of two nodes a_1, a_2 as the quotient $v(a_1,a_2) = \dfrac{\|\vec{x}(a_1) - \vec{x}(a_2)\|}{t(a_1) - t(a_2)}$.
Using this formalism, we introduce th discrete Lorentz-group as the group of (Lorentz-group of unitary rotations) spatial rotations and boosts of the discrete Lorentzian space *S(1,n)* with maximum velocity *c* .
For two nodes (=physical events) a_1, a_2 the node a_2 follows (is in the future of) a_1, if there is a graph path leading from a_1 to a_2, then $t(a_1) < t(a_2)$ is fulfilled automatically . This future-relation in time $t(a_1) < t(a_2)$ is a partial time-ordering in the graph .

Theorem
For a causal invariant acyclic binary (2-relation) HTA *h* embedded in Lorentz space, the future-relation is invariant under Lorentz-rotations.

2.2 Characterization of HTA

-Dimension

Volume of a HTA $V_r(r,n)$ within radius r and after n steeps is the number of nodes within n neighborhoods from the starting point. For a *d*-dimensional grid we have in the limit $n \to \infty$ [8]

$V_r(r) = V_r(r,\infty) = 2r+1 \quad d=1$

$V_r(r) = 2r^2 + r + 1 \quad d=2 \quad Vr=4r33+2r2+8r3+1$

$V_r(r) = \frac{4r^3}{3} + 2r^2 + \frac{8r}{3} + 1 \quad d=3$

In general, the dimension of a HTA depends on it rule and can be calculated in the limit $n \to \infty$ as

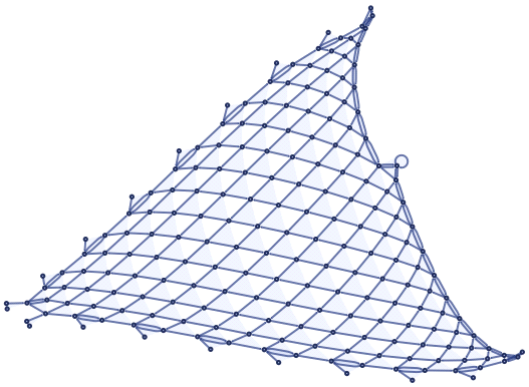
$\Delta(r) = \lim\left(\frac{log\left(V_r(r+1,n)\right) - log\left(V_r(r,n)\right)}{log(r+1) - log(r)} \mid n \to \infty \right)$ where $\Delta(r,n) = \frac{log\left(V_r(r+1,n)\right) - log\left(V_r(r,n)\right)}{log(r+1) - log(r)}$

Example [11]

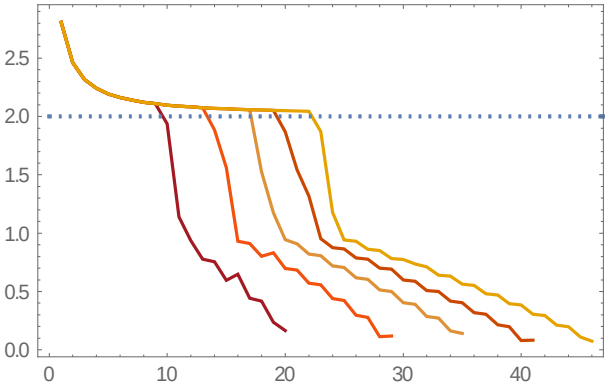
$2_3 \to 3_3$ rule $\{\{1,2,2\},\{3,1,4\}\} \to \{\{2,5,2\},\{2,3,5\},\{4,5,5\}\}$

initial state $\{\{1,1,1\},\{1,1,1\}\}$

HTA after n=200 steps



The dimension $\Delta(r,n)$ is



and in the limit $\Delta(r)$=2

-Curvature [11]

The volume of the *d*-dimensional ball with radius *r* in curved space with curvature *c* is

$V_b(d,c,r) = \frac{\pi^{d/2}}{(d/2)!} r^d \left(1 - \frac{r^2}{6(d+2)} c + O(c^2 r^4)\right)$, in flat space $V_b(d,r) = \frac{\pi^{d/2}}{(d/2)!} r^d$

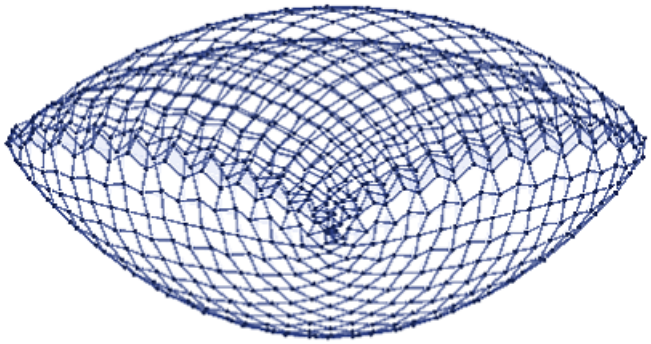
so the curvature introduces a negative correction to volume in order r^2 .

Example [11]

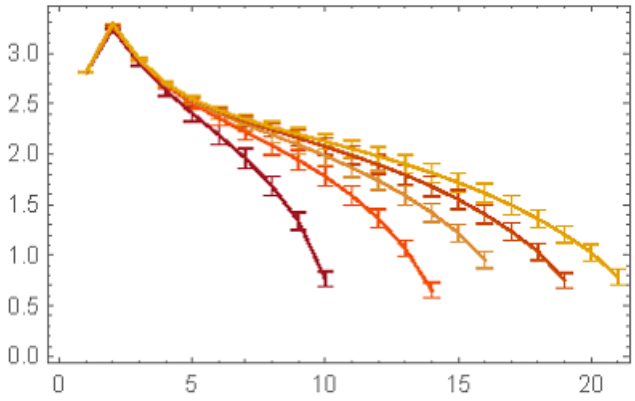
$2_3 \to 3_3$ rule $\{\{1,2,2\},\{4,2,5\}\} \to \{\{6,3,1\},\{3,6,4\},\{1,2,6\}\}$

initial state $\{\{1,1,1\},\{1,1,1\}\}$

HTA after n=800 steps



The dimension $\Delta(r,n)$ is



-Local volume and dimension

The local volume $V_r(r,n,x)$ and local dimension $\Delta(r,n,x)$ around a node x reflects the local structure of a HTA .

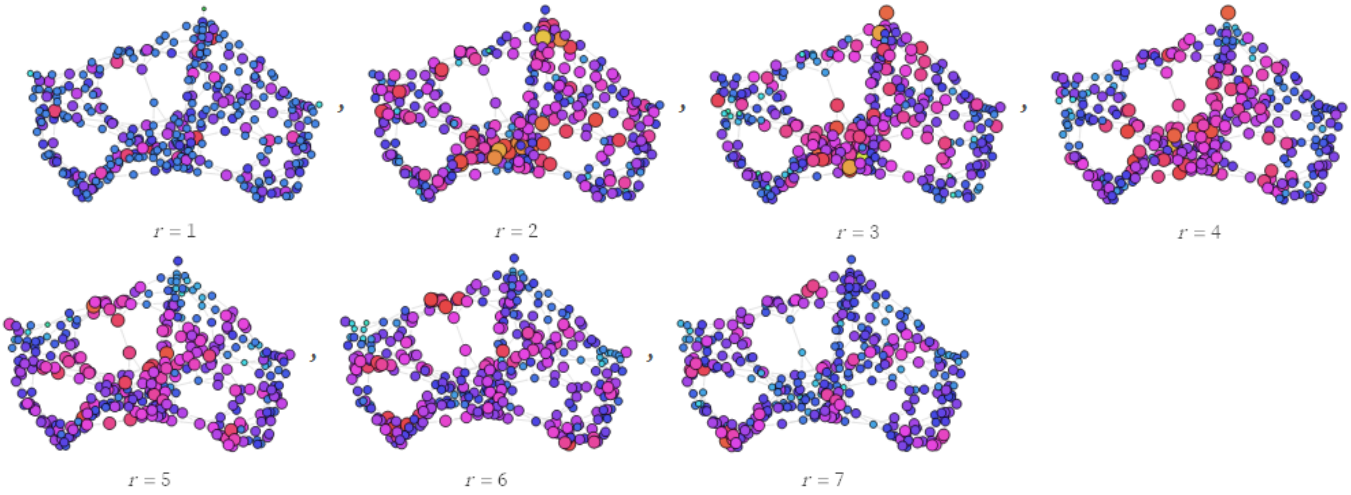
Example [11]

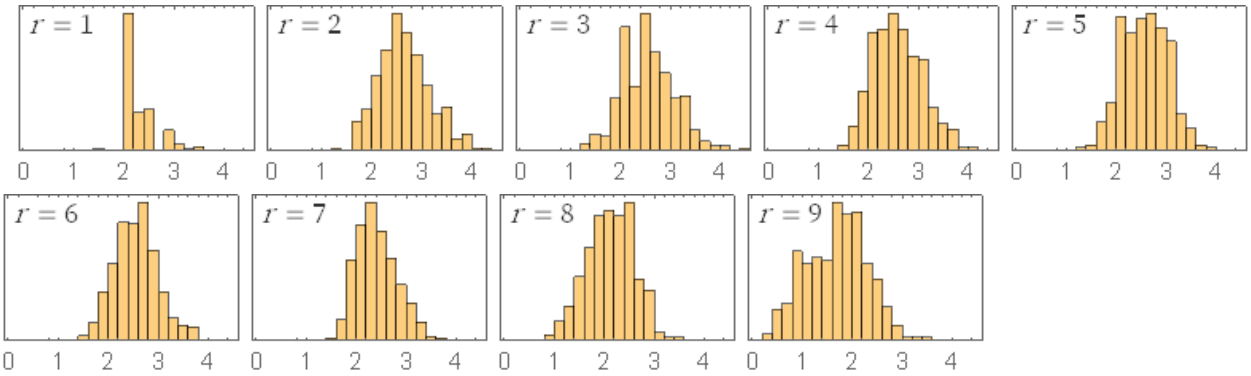
$2_2 \to 4_2$ rule $\{\{1,2\},\{1,3\}\} \to \{\{1,3\},\{1,4\},\{2,4\},\{3,4\}\}$

initial state $\{\{1,2\},\{1,3\}\}$

HTA after n=10 steps

the distribution of local dimension $\Delta(r,n,x)$ in the graph and as histogram is





-Connectivity

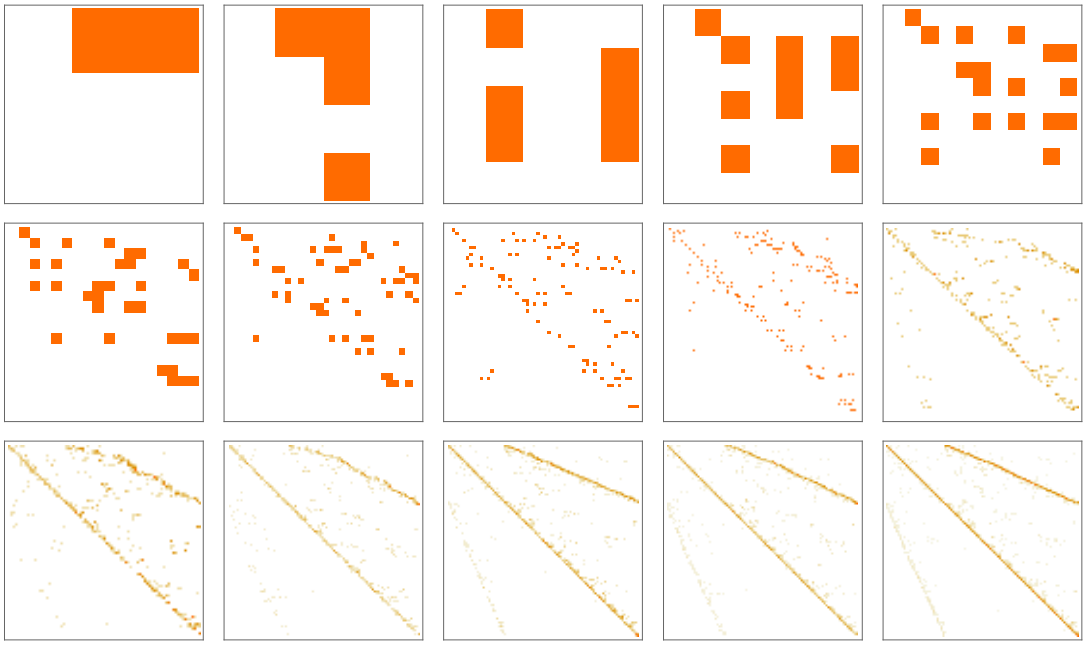
A measure of connectivity is the adjacency matrix $A(x_1, x_2) = \#\{e(x_1, x_2)\}$, i.e. number of directed edges from one node to another node

Example [11]

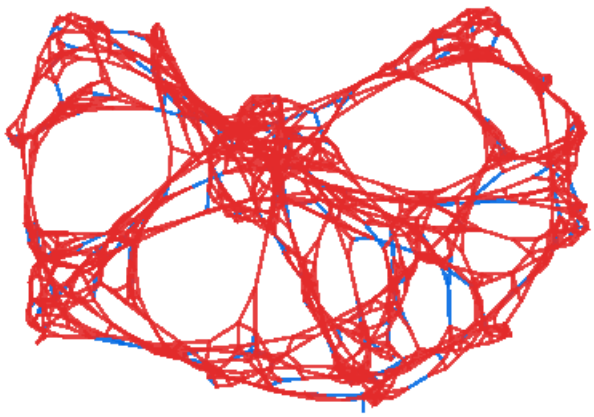
$2_2 \to 4_2$ rule $\{\{1,2\},\{1,3\}\} \to \{\{1,3\},\{1,4\},\{2,4\},\{3,4\}\}$

initial state $\{\{1,2\},\{1,3\}\}$

$A(x_1, x_2)$ after *n=1,...,15* steps

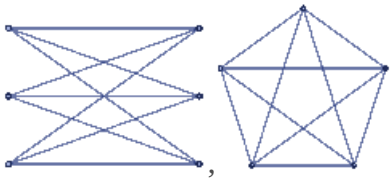


The age of an edge is its lifetime at step number *n* is $t(e) = n - n(e)$ the difference between the final time *n* and its birth time $n(e)$, the age distribution at *n=12*, from low=blue to high=red is

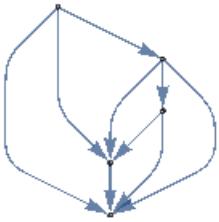


-Planarity

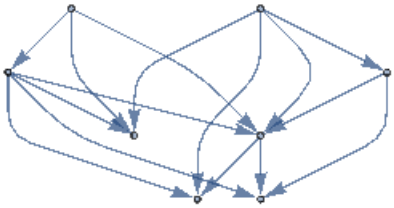
It is known that a graph is planar exactly if it does not contain subgraphs $K_{3,3}$ and K_5 (Wagner's theorem [10]).



HTA with rules $1_2 \to 2_2$ and $1_2 \to 3_2$ preserve planarity, but the above $2_2 \to 4_2$ rule



$\{\{1,2\},\{1,3\}\} \to \{\{1,3\},\{1,4\},\{2,4\},\{3,4\}\}$ does not, it transforms the planar graph



into the non-planar graph

2.3 Causal invariant HTA

The simplest causal invariant HTA are those with rule with left-hand-side consisting of one element only, e.g. $1_2 \to 2_2$ rule $\{\{1,2\}\} \to \{\{1,2\},\{2,3\}\}$

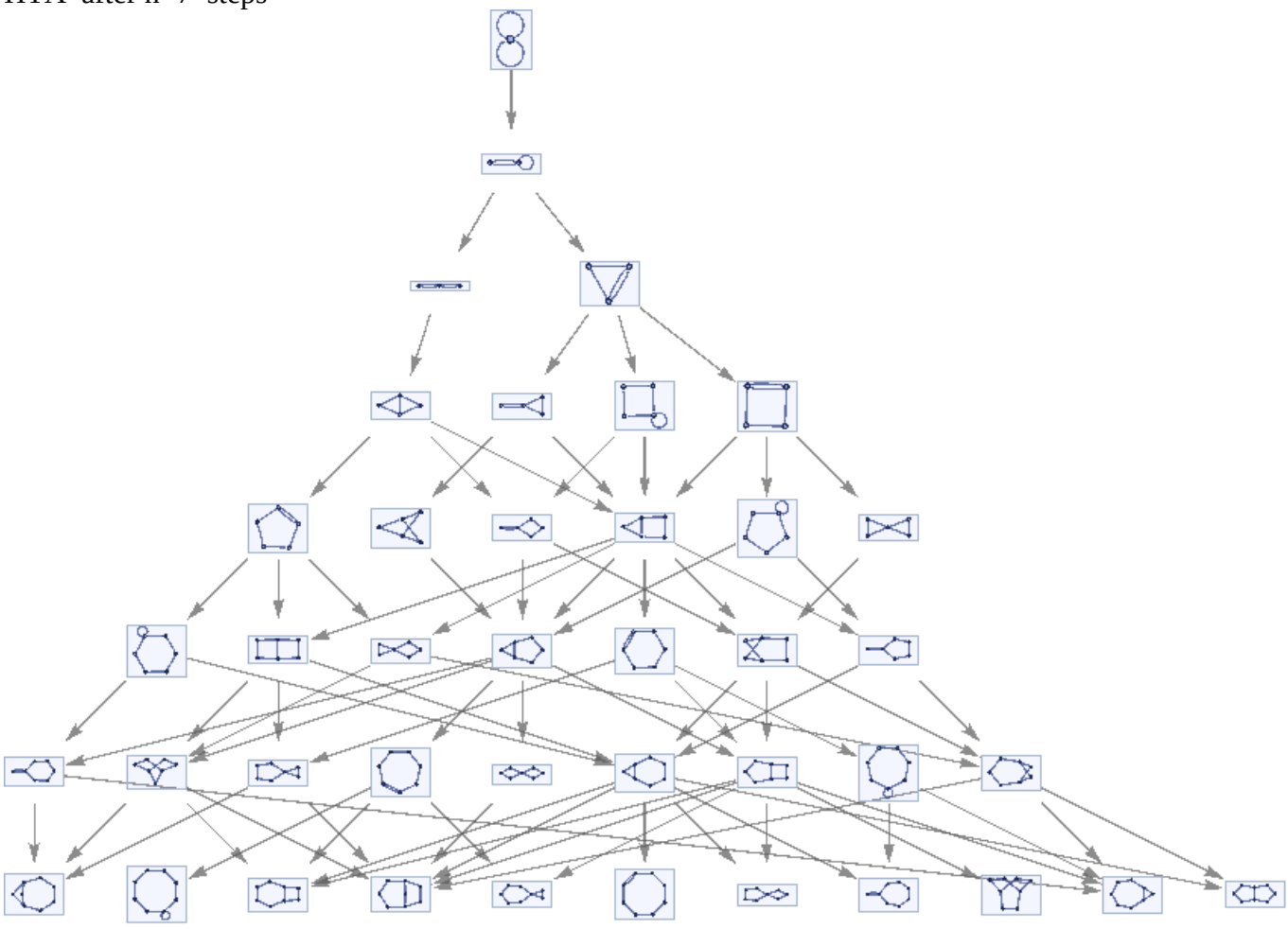
Example causal invariant HTA, convergent in 3 steps [11]

A more complicated causal invariant rule is

$2_2 \to 3_2$ rule $\{\{1,2\},\{1,3\}\} \to \{\{2,4\},\{2,3\},\{4,1\}\}$

initial state $\{\{1,1\},\{1,1\}\}$

HTA after n=7 steps



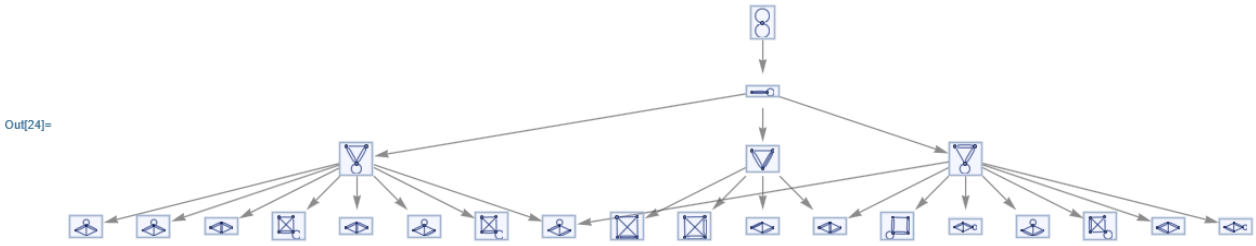
this graph is causal invariant, with branches converging in at most 3 steps, the node number increases with $N(n) \sim n^{3/2}$

Example non-causal-invariant HTA [11]

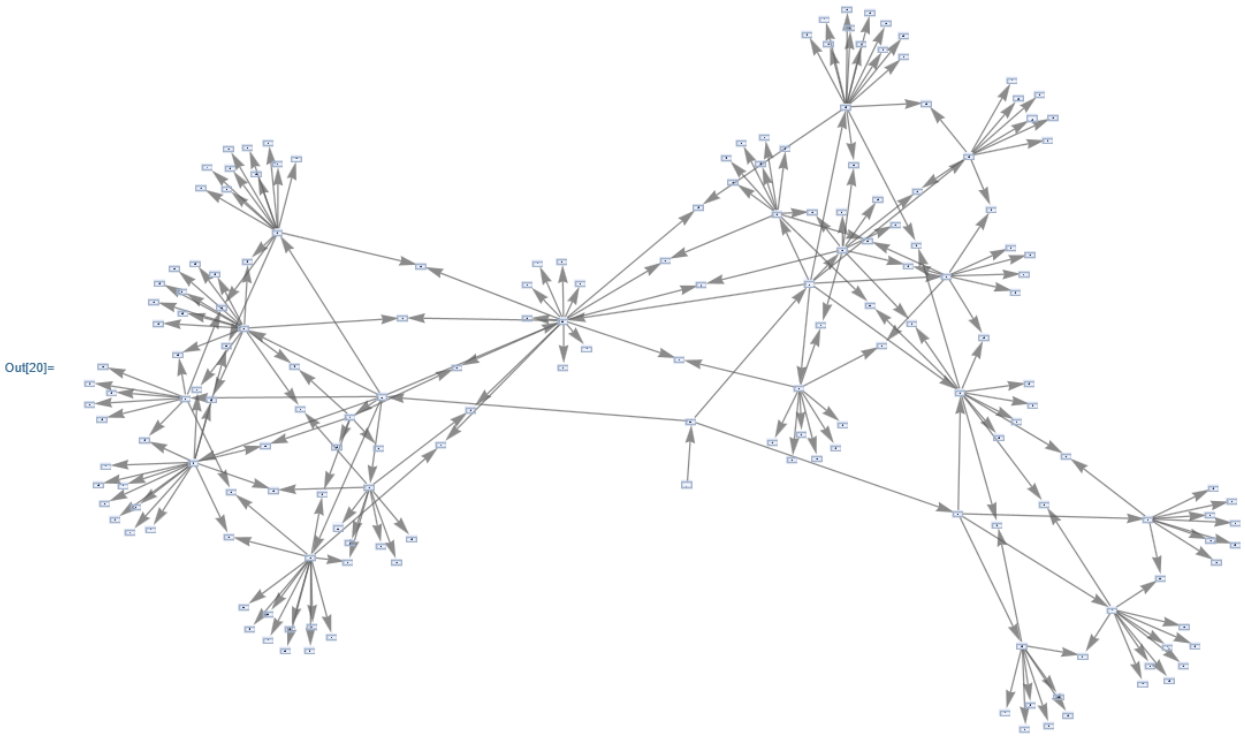
$2_2 \to 4_2$ rule $\{\{1,2\},\{1,3\}\} \to \{\{1,2\},\{1,4\},\{2,4\},\{3,4\}\}$

initial state $\{\{1,1\},\{1,1\}\}$

HTA after n=3 steps



HTA after n=4 steps



the node number increases with $N(n) \sim 3^n$

Conjecture: causal invariant HTA has a node number increasing at most polynomially $N(n) \sim n^p$, and it converges in *k* steps, where $k \approx 2p$

2.4 Symmetries in HTA

Symmetries of rules in HTA are reflected in conservation properties of the ensuing graphs.

For HTA with rules with ternary relations, we get 6 symmetry classes , because the 3–permutation group S_3 has 6 subgroups:

-identity

-3 cases of the 2–permutation group S_2 : {1,3,2}, {3,2,1}, {2,1,3}

-2 cases of cyclic rotation A_3 : {2,3,1} , {3,1,2}

-full group S_3

Normally, one considers number-permutations of the left side (left-invariance). In some cases, also position-permutations are considered, normally on the right side.

The *identity criterion* is the *Wolfram-normal-form*: a rule is renumbered in all possible ways and the renumbered rules are string-sorted lexicographically, the first is taken as the Wolfram-normal-form.

Example rule permutation (see example 3 below)

rule=$\{\{1,2,3\},\{2,3,1\}\} \to \{\{2,2,4\},\{4,3,3\},\{1,4,1\}\}$

Wnf(rule)=rule=$\{\{1,2,3\},\{2,3,1\}\}\to\{\{2,2,4\},\{4,3,3\},\{1,4,1\}\}$

permutation $3\to 2\to 1$ yields prule=$\{\{3,1,2\},\{1,2,3\}\}\to\{\{2,2,4\},\{4,3,3\},\{1,4,1\}\}$

Wnf(prule)= $\{\{1,2,3\},\{2,3,1\}\}\to\{\{3,3,4\},\{4,1,1\},\{2,4,2\}\}$ $\neq$ Wnf(rule)

Multiplication table of S_3

	$()$	$(1,2)$	$(2,3)$	$(1,3)$	$(1,2,3)$	$(1,3,2)$
$()$	$()$	$(1,2)$	$(2,3)$	$(1,3)$	$(1,2,3)$	$(1,3,2)$
$(1,2)$	$(1,2)$	$()$	$(1,2,3)$	$(1,3,2)$	$(2,3)$	$(1,3)$
$(2,3)$	$(2,3)$	$(1,3,2)$	$()$	$(1,2,3)$	$(1,3)$	$(1,2)$
$(1,3)$	$(1,3)$	$(1,2,3)$	$(1,3,2)$	$()$	$(1,2)$	$(2,3)$
$(1,2,3)$	$(1,2,3)$	$(1,3)$	$(1,2)$	$(2,3)$	$(1,3,2)$	$()$
$(1,3,2)$	$(1,3,2)$	$(2,3)$	$(1,3)$	$(1,2)$	$()$	$(1,2,3)$

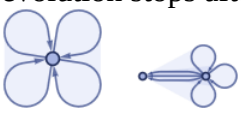
Example HTA S_3 fully left-invariant [11]

$2_3\to 3_3$ rule $\{\{1,2,3\},\{2,3,1\}\}\to\{\{1,1,4\},\{2,2,4\},\{3,3,4\}\}$

initial state $\{\{1,1,1\},\{1,1,1\}\}$

HTA after n=0,1,...,7 steps

evolution stops after n=1 steps

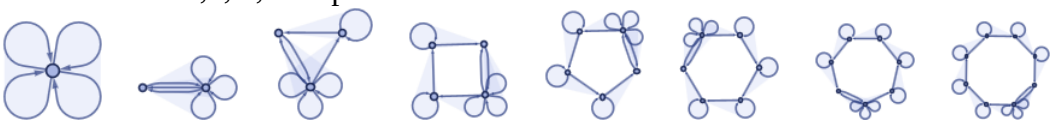


Example HTA $S_2(13)$ left-invariant [11]

$2_3\to 3_3$ rule $\{\{1,2,3\},\{2,3,1\}\}\to\{\{4,2,2\},\{3,3,4\},\{1,1,4\}\}$

initial state $\{\{1,1,1\},\{1,1,1\}\}$

HTA after n=0,1,...,7 steps



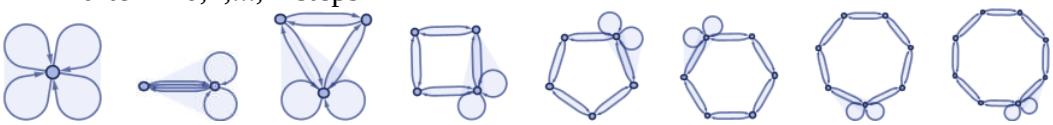
The resulting graph after *n* steps is a *n*-polygone with *n-1* loops and one threefold loop.

Example HTA S3 left number-permutation + right position-permutation invariant [11]

$2_3\to 3_3$ rule $\{\{1,2,3\},\{2,3,1\}\}\to\{\{2,2,4\},\{4,3,3\},\{1,4,1\}\}$

initial state $\{\{1,1,1\},\{1,1,1\}\}$

HTA after n=0,1,...,7 steps



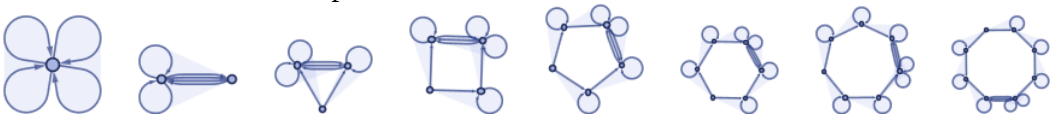
The resulting graph after *n* steps is a *n*-polygone with a double-loop in different positions.

Example HTA S3 left number-permutation + right position-permutation invariant [11]

$2_3\to 3_3$ rule $\{\{1,2,3\},\{2,3,1\}\}\to\{\{4,4,2\},\{4,1,4\},\{3,4,4\}\}$

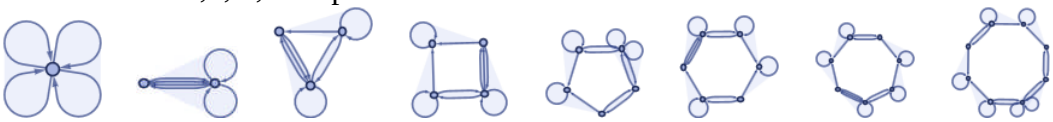
initial state $\{\{1,1,1\},\{1,1,1\}\}$

HTA after n=0,1,...,7 steps



The resulting graph after *n* steps is a *n*-polygone with *n-1* loops and a double-loop.

Example HTA S3 left number-permutation + right position-permutation invariant [11]
$2_3 \to 3_3$ rule $\{\{1,2,3\},\{2,3,1\}\} \to \{\{1,1,4\},\{4,2,2\},\{3,4,3\}\}$
initial state $\{\{1,1,1\},\{1,1,1\}\}$
HTA after n=0,1,...,7 steps



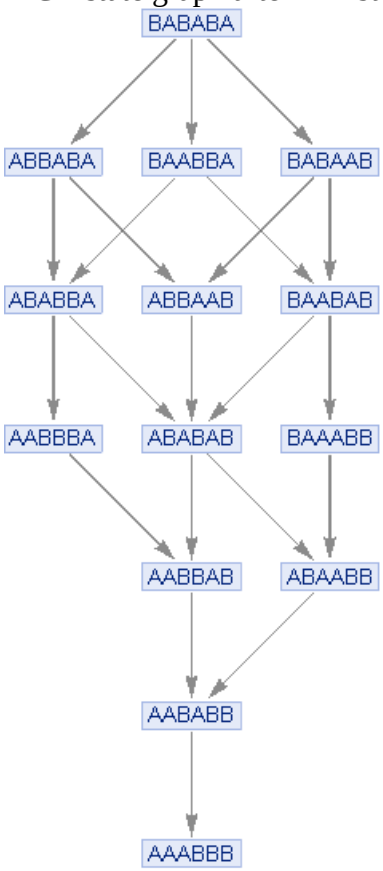
The resulting graph after *n* steps is a *n*-polygone with *n-3* loops and a double-loop.

3. Properties of MCA

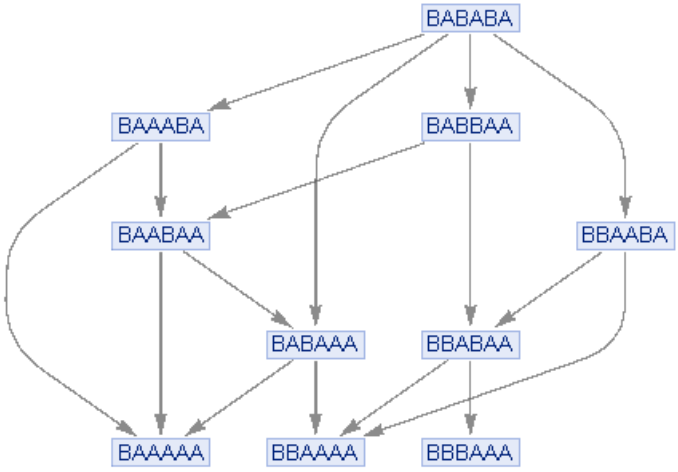
3.1 MCA and their connections to physics

-Causal invariance

MCA *h* is called *causal invariant*, if its output after a certain number of transitions is equal under arbitrary sequences, i.e. it converges to a fixed path-independent configuration.
Example causal invariant MCA [11]
$2 \to 2$ rule $\{BA \to AB\}$
initial state *BABABA*
MCA state graph after n=7 steps



where all path converge to a single string *AAABBB* , so MCA is causal invariant.
Example non-causal-invariant MCA [11]
$2 \to 2, 2 \to 2$ rule $\{AB \to AA, AB \to BA\}$
initial state *BABABA*
MCA state graph after n=7 steps



here the terminal configuration, which does not change anymore, consists of three strings $\{BAAAAA, BBAAAA, BBBAAA\}$, depending on the path, so MCA is not causal invariant

The finite convergence behavior of causal invariant MCA varies widely in number steps, size of graph, and intermediate strings, the following table gives an overview

	rule	intermediate strings	number of nodes
1	{A → A}	A → {A, A} → A	2
2	{A → B, AB → AA}	AB → {BB, AA} → BB	5
3	{A → AA, A → BAB}	A → {AA, BAB} → BABABAB	22
4	{A → AA, A → BAAB}	A → {AA, BAAB} → BAABAABAAB	121
5	{A → AA, A → BAABB}	A → {AA, BAABB} → BAABBAABBAABBABB	515
6	{A → AAB, ABAA → A}	ABAA → {AABBAA, A} → AABBAABB	1664
7	{A → AAB, ABBA → A}	ABBA → {AABBBA, A} → AAAABBBAAB	2401
8	{A → AA, AA → BABBB}	AA → {AAA, BABBB} → BABBBABBBABBBBBBBB	759
9	{A → B, BB → A, AAAA → B}	AAAA → {BAAA, B} → B	30
10	{A → AA, AA → BABBBB}	AA → {AAA, BABBBB} → BABBBBABBBBABBBBBBBBBBBBBB	4020
11	{A → AA, AAA → BABBB}	AAA → {AAAA, BABBB} → BABBBABBBABBBBBBBB	2294
12	{A → AA, AA → B, BBBB → A}	AA → {AAA, B} → B	405
13	{A → B, BB → A, AAAAAB → A}	AAAAAB → {BAAAAB, A} → A	94
14	{A → AB, BAA → A, BBB → A}	BBBAA → {AAA, BBA} → AA	2698
15	{A → AA, BBB → A, AAAA → B}	AAAA → {AAAAA, B} → B	430
16	{A → AA, AAA → B, BBBB → A}	AAA → {AAAA, B} → B	906

Conjecture: fraction of causal–invariant MCA in n steps decreases like 2^{-n} .

-Causal invariance with infinite growth

Causal MCA graphs do not always evolve to a definite final state after a finite number of steps, there are infinite causal invariant MCA graphs.

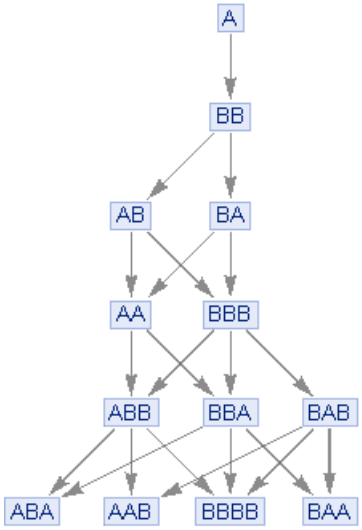
Example infinite causal invariant MCA [11]

```
[{"A" → "BB", "B" → "A"}, "A", 5, "StatesGraph"]
```

$1 \to 2, 1 \to 1$ rule $\{A \to BB, B \to A\}$

initial state A

MCA state graph after n=5 steps



This MCA is causal invariant, but it does not terminate, it keeps growing forever.

3.2 Characterization of MCA

-Multiplicity

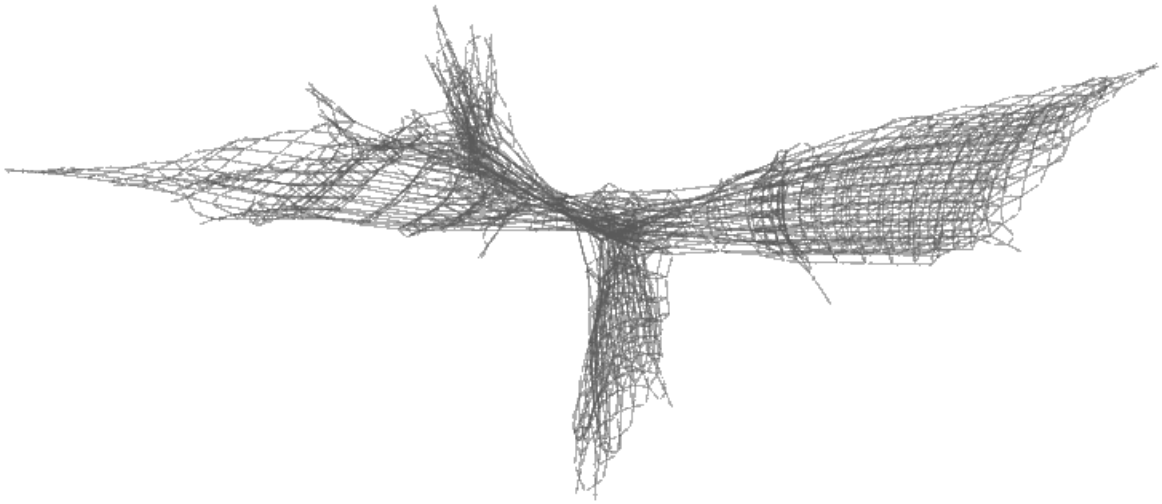
In analogy to the volume $V_r(r,n,x)$ of HTA, we define the *multiplicity* of MCA state graph $M_t(n,s)$ as the number of distinct strings after *n* steps evolving from an initial string *s* . Rules yield either asymptotically exponential or asymptotically polynomial M_t .

Example polynomial multiplicity [11]

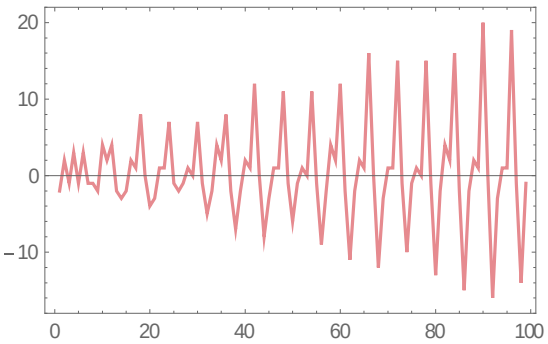
$2 \to 1, 3 \to 6$ rule $\{AB \to A, ABA \to BBAABB\}$

initial state $ABAAA$

MCA state graph after n=25 steps



and $M_t(n,s) \sim n^2$ and quasiperiodic



We define the local multiplicity of MCA evolution graph $C_t(n,s,b,x)$ as the number of distinct strings after n steps evolving from an initial string s, for a neighborhood b around node-string x. In general, its behavior for large n is polynomial with the fractal deimension d: $C_t(n,s,b,x) \sim n^d$

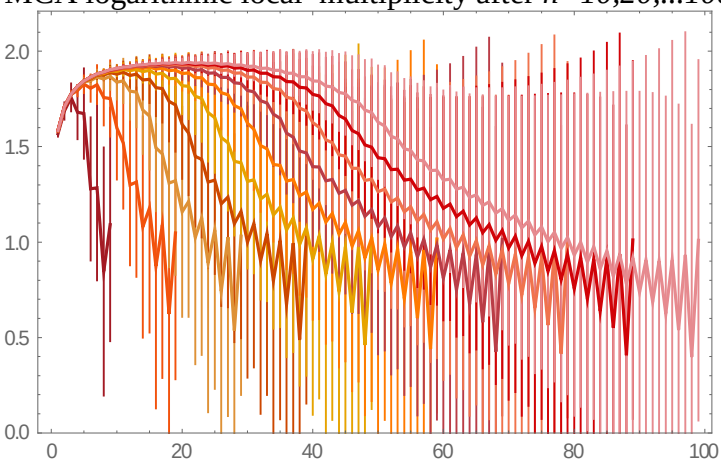
Example local multiplicity [11]

```
[{"A" → "AB", "BB" → "BB"}, "A", t]
```

$1 \to 2, 2 \to 2$ rule $\{A \to AB, BB \to BB\}$

initial state A

MCA logarithmic local multiplicity after $n=10,20,...100$ steps, averaged over neighborhoods as function of b



for $b<<n$, the dimension is $d=2$, as expected for a planar graph

-Ordering

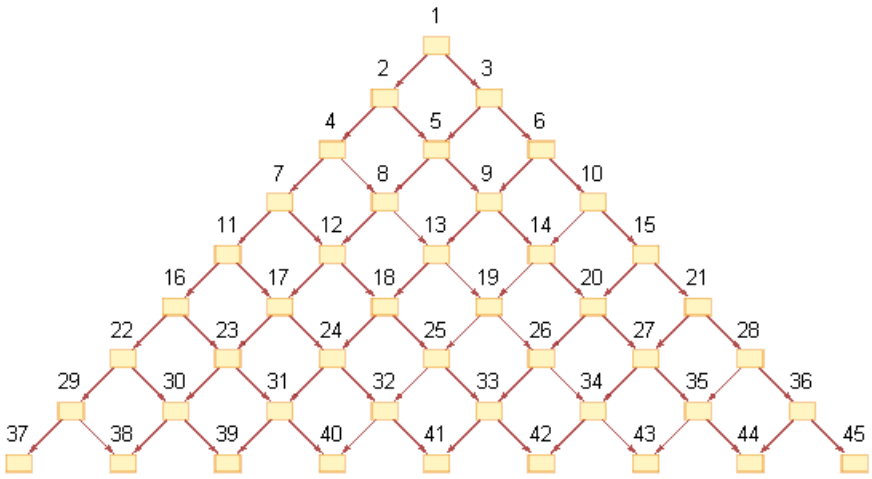
A causal MCA graph can be ordered, there are two common schemes consistent with causal invariance: breadth-first and depth-first .

Example ordering [11]

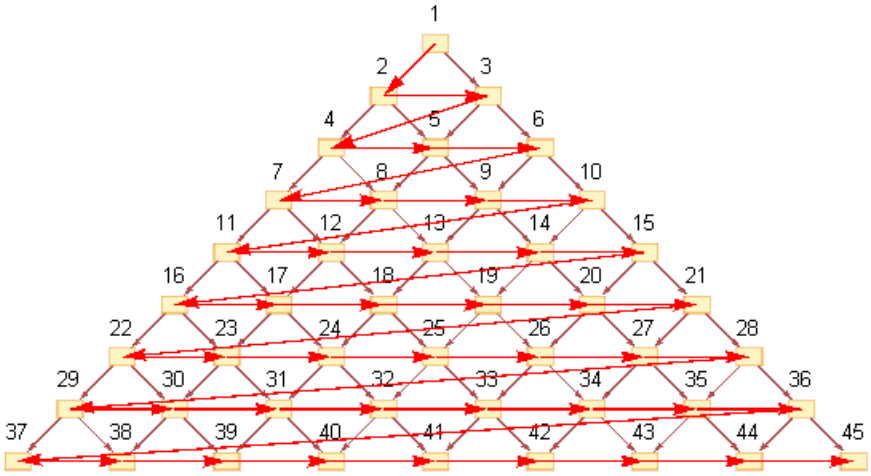
$1 \to 2, 2 \to 2$ rule $\{A \to AB, BB \to BB\}$

initial state A

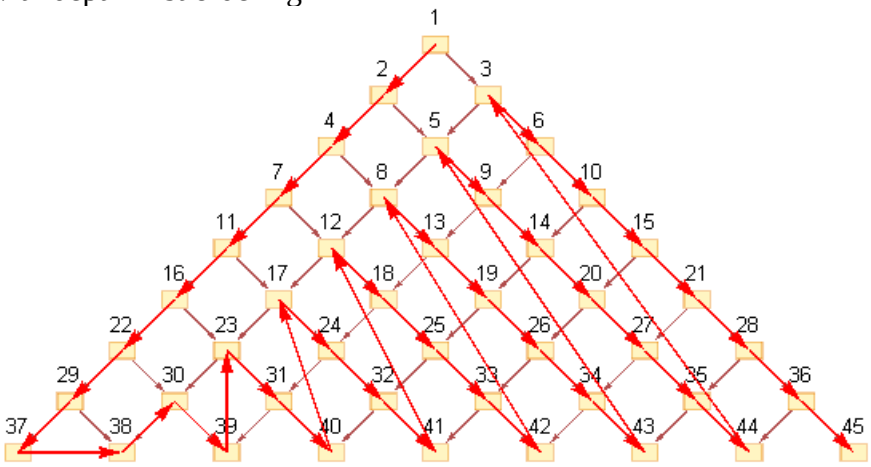
MCA graph after n=9 steps



with breadth-first ordering



with depth-first ordering



-Branchial graph

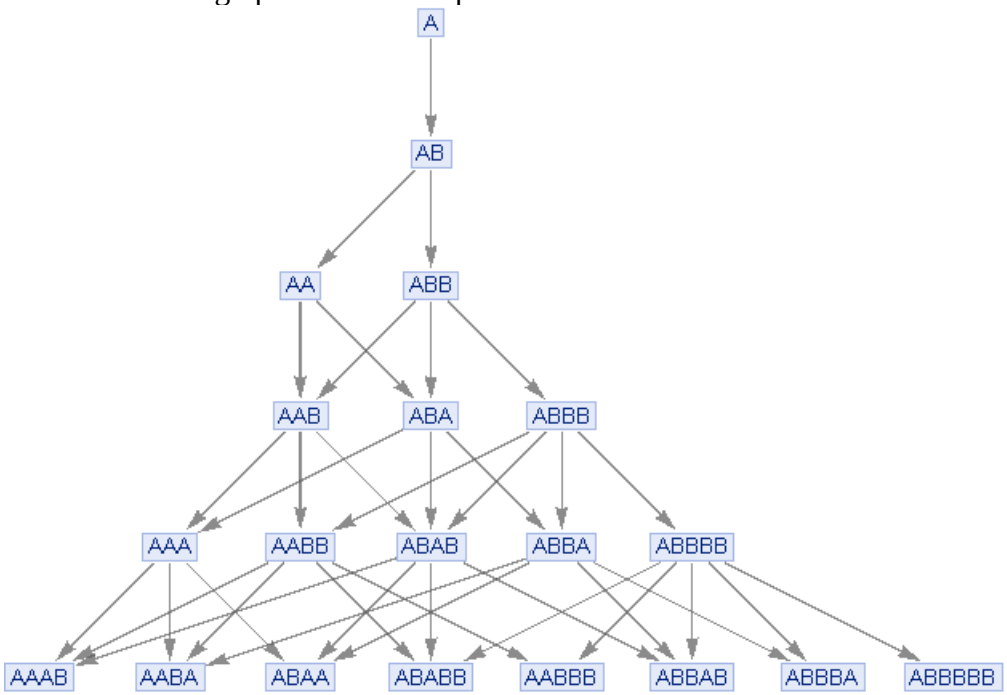
The *branchial graph* of a causal MCA graph is formed by connecting nodes on the same level in a causal graph.

Example branchial graph [11]

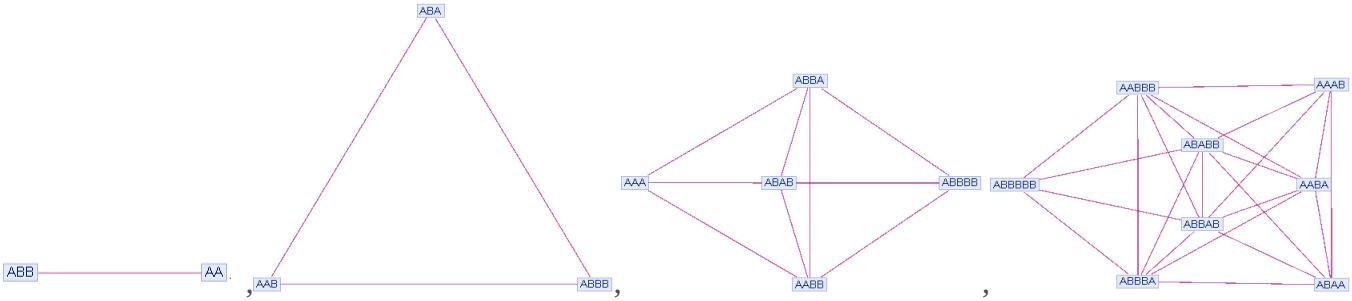
$1 \to 2, 1 \to 1$ rule $\{A \to AB, B \to A\}$

initial state A

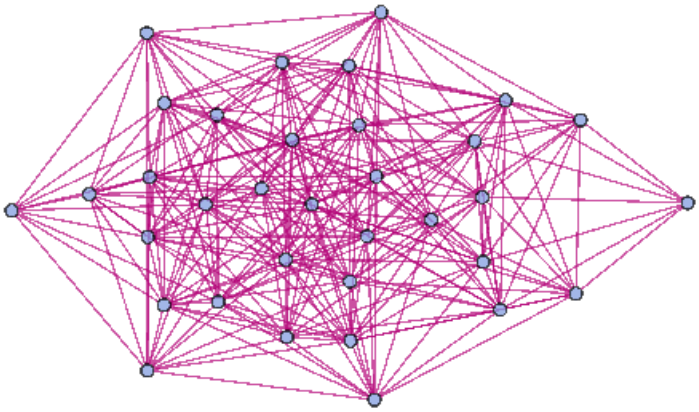
MCA evolution graph after n=5 steps



the branchial graphs at consecutive steps n=2,...5 are

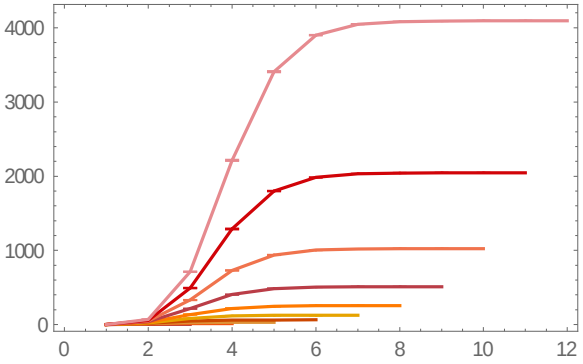


for n=9



The *connectivity* of a graph is measured by $B_b(n,b,x)$, which is the number of nodes at step *n*, within a distance *b* from a given node *x* .

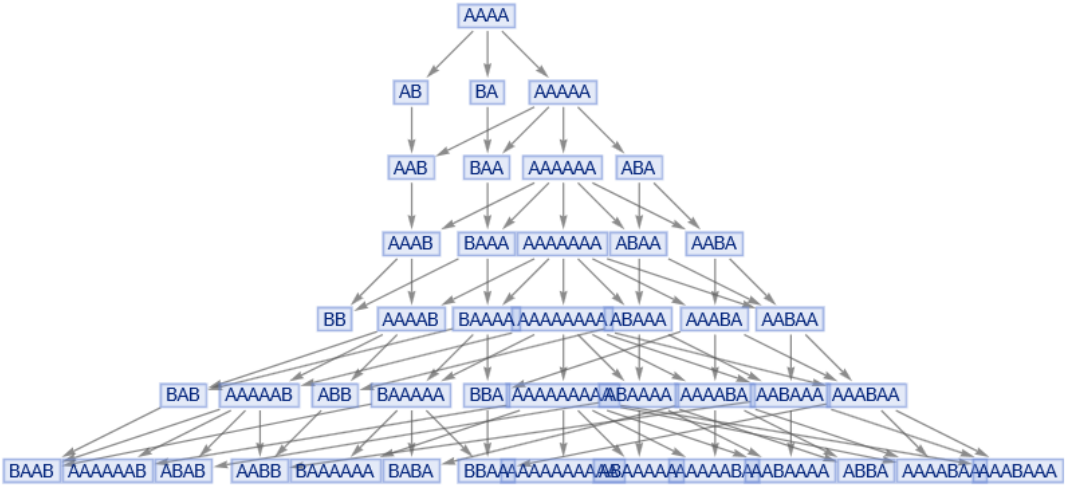
Example connectivity [11]
$0 \to 1, 0 \to 1$ rule $\{\to A, \to B\}$
initial state empty
connectivity $B_b(n,b,x_1)$ after *n* steps as a function of *b*



4 Evolution of MCA with genetic algorithm with goal=entropy

4.1 Initialization and goal function

We start with a MCA (multiway cellular automaton) with the string replacement rule [11]
rmgr1={"A"→"AA","AAA"→"B","BBBB"→"A"} with 3 components
and the initial string *imgr1*="AAAA" .
The rule and the initial string can be transformed into
rule string list = {rule source strings, rule replacement strings, initial string}
{"A","AAA","BBBB","AA","B","A","AAAA"}, with here 7=2*3+1 components,
which has the entropy =2.875, calculated according to Part1 3.3 .
we get the binary form of it with A=0, B=1 .
This rule generates the evolution graph after *nstep=6* steps *mgr1=*



we can interpret the graph as a two-dimensional binary image with A=0, B=1, where the rows are the strings of one step, the columns are the steps, *n=0, 1, ...6*.
In this way, we can form the morphological components and calculate the entropy of the graph, like in Part2 with cellular automata.
We have then the image

with the morphological components

and the graph entropy= 9.625 .

We set the goal function to be the graph entropy, i.e. the initial value is 9.625.

4.2 Algorithm

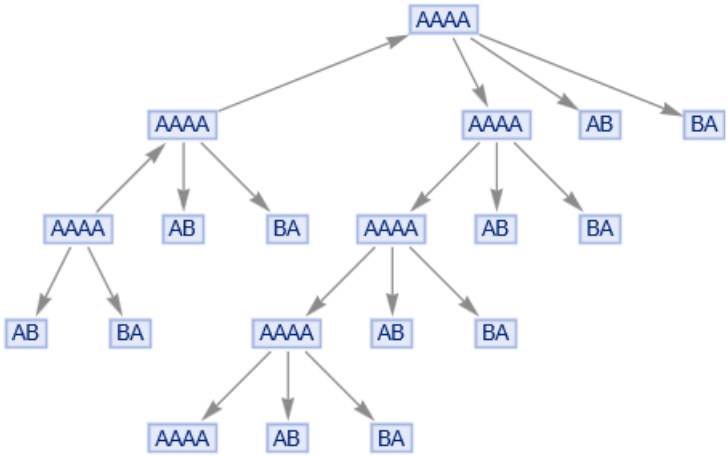
The genetic algorithm uses a pool of *ngen* (here *ngen=10*) rules, which form a generation, and in each step the generation members are cross-bred, mutated, and reordered according to goal function: the first *nbest* (*nbest=4*) generate the remaining *ngen-nbest* members to form the new generation.
The genetic algorithm starts with a rule and initial string, here *rmgr1* and *imgr1* .
In the initialization step *n=0*, the rule string list is mutated with random list index *indrl*, random string position *indstr*, and random mutation operation number *opindex*=1,...8 , which add *ngen-1* mutated rules.
We use 8 mutation operations on a string *str* in string position *pos* in the form
operstr(str, pos)={inversion(A−>B,B−>A), permutation_left, permutation_right, insertion_right_A, insertion_right_B, insertion_left_A, insertion_left_B, deletion}

In an iteration step, the following procedures are executed:
▪ crossing the first *nbest* (here *nbest=4*) to generate *ngen-nbest* new rules
▪ the new rules replace the worst *ngen-nbest* members

▪ the new *ngen-nbest* members are mutated randomly
▪ the new generation={*nbest* old members, *ngen-nbest* new members }
▪ the new generation evolution graphs and graph entropies are calculated
▪ the generation is sorted according to graph entropy

4.3 Algorithm results

After *nastep=10* iterations, we get a very low entropy of 0.0029, and a simple evolution graph
the best rule (=first in the sorted list *lrulelist*) is *rulee=lrulelist[1]* :
rule string list *rulee= {"AA","AAA","BBBB","AA","B","A","AAAA"}*
rule *rmgr1e={"AA"→"AA","AAA"→"B","BBBB"→"A"}*
with the graph *mgr1e=*



with the graph image

and the entropy=0.0029 .

The rule *rmgr1e* differs from the initial rule *rmgr1* only in the first component
"AA"→"AA" replaces *"A"→"AA"*
and the resulting evolution graph stops after 3 steps and does not evolve further, which yields a simple pattern and correspondingly low entropy.

References

[1] S. Wolfram, A new kind of science, Wolfram Media, 2002
[2] S. Wolfram, , Cellular automata and complexity, CRC Press, 2018
[3] J. Helm, Mathematica-notebook: Cellular automata CellAutom.nb, www.researchgate.net, 2021
[4] S, Nichele et al., Genotype Regulation by Self-Modifying Instruction-Based Development on Cellular Automata, Parallel Problem Solving from Nature , Springer International, 2016
[5] K. C. Clarke et al., A self-modifying cellular automaton, Environment and planning, 1997
[6] J. Helm, Mathematica-notebook: Turing machines TuringMachine.nb, www.researchgate.net, 2021
[7] H. Steinitz, Kolmogorov complexity and algorithmic randomness, University of Chicago, 2013
[8] S. Wolfram, Physics project, Wolfram Media, 10/2020
[9] S. Wolfram, Introducing Multicomputation as a Fourth General Paradigm for Theoretical Science, Wolfram Media, 09/2021
[10] J. Gorard, Some Relativistic and Gravitational Properties of the Wolfram Model, Wolfram Media, 04/2021
[11] J. Helm, Mathematica-notebook: Multiway cellular automata and Hypergraph transition automata CellAutomGraph.nb, www.researchgate.net , 2022